Density based clustering methods

In this practice you will:
- implement the clustering method presented in the paper: Rodriguez, Alex, and Alessandro Laio. "Clustering by Fast Search and Find of Density Peaks." Science 344, no. 6191 (2014): 1492–96. https://doi.org/10.1126/science.1242072.
- apply it to the MNIST dataset
- apply the DBSCAN method on the MNIST dataset
- learn how clustering performances with these two methods are affected by the parameters of the algorithms

0. Import libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans #import KMeans algorithm
from sklearn.cluster import DBSCAN #import DBSCAN algorithm
from sklearn.preprocessing import StandardScaler #import algorithm to standardize
data
from sklearn.metrics.pairwise import pairwise_distances #algorithm to compute all
distances between points
from sklearn.datasets import make_blobs #function used to generate sinthetic
dataset
from sklearn.metrics import adjusted_rand_score #ARI
```

1. Create a synthetic dataset

In [ ]:
```
mean1 = [1, 6] #define the center of the first cluster
cov1 = [[0.5, 0.0],
        [0.0, 0.5]] #covariance matrix of the first cluster
X1 = np.random.multivariate_normal(mean1, cov1, 120) #generate 120 random samples
distributed using a Gaussian using mean and cov
mean2 = [9, 7] #=
cov2 = [[2.5, 1.8],
        [1.8, 1.5]]
X2 = np.random.multivariate_normal(mean2, cov2, 180)
mean3 = [-4, 4] #=
cov3 = [[4.2, -2.4],
        [-2.4, 4.0]]
X3 = np.random.multivariate_normal(mean3, cov3, 100)
X = np.vstack([X1, X2, X3]) #stacks vertically the 3 matrices building a unified
dataset of 400 rows and 2 columns
y_true = np.array([0]*len(X1) + [1]*len(X2) + [2]*len(X3)) #create ground truth,
120 0, 180 1, 100 2
feature_names = ['feat_1', 'feat_2'] #names for the features
scaler = StandardScaler() #initialize the standardization tool
X_scaled = scaler.fit_transform(X) #firstly it computes mean and std dev and then
transforms data subtracting the mean and dividing by the std dev
pd.DataFrame(X_scaled, columns=feature_names).describe().loc[["mean", "std"]]
#convert the std matrix in pandas df, compute statistic and isolate mean and std
dev
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    s=25, #dot dimensions
    alpha=0.7 #transparency
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title("Synthetic dataset")
plt.show()
```

2. Clustering based on density peaks

Write a function to calculate for each point in the dataset:
* the local density, **rho**, defined as the number of samples within a given distance epsilon;
* the index of the closest sample with higher density, **inds_closest_higher_rho**. For the sample with highest density set this value to -1;
* the distance to the closest sample with higher density, **delta**.

The function should returns the arrays rho, inds_closest_higher_rho, and delta, and it should plot delta as a function of rho. Test the function using the synthetic dataset, and values of epsilon equal to 0.1, 1, and 10. How does the plot change ? What is the optimal value ?

In [ ]:
```
def density_peaks(X, eps): #define density peak function
    D = pairwise_distances(X) #compute a matrix of L2 distances between points
    rho = np.sum(D < eps, axis = 1) #compute a boolean matrix of y/n, sums the
values per row, compute the density locally for each point
    inds_maxmin_rho = np.argsort(rho)[::-1] #order the points in decreasing order
and and the first one is the most dense
    delta = np.zeros(len(rho)) #initialize a zeros array to store distances
between each point and the closest high density point to that point
    delta[inds_maxmin_rho[0]] = np.max(D) #for the highest density point we assign
the highest distance computed
    inds_closest_higher_rho = -1*np.ones(len(rho), dtype = int) #initialize a -1s
array with all the "fathers density" of each point
    for ind in range(1, len(rho)): #from the highest density point to the lowest
        ind_sample = inds_maxmin_rho[ind] #takes the index of the actual point
```

```
        inds_higher_density = inds_maxmin_rho[:ind] #isolate all the points with
an highest density than the actual point
        inds_closest_higher_rho[ind_sample] = inds_higher_density[ #compute the
distance between the actual point and all the higher density points , finds the
closest one and saves it
            np.argmin(D[ind_sample, inds_higher_density])
        ]
        delta[ind_sample] = np.min(D[ind_sample, inds_higher_density]) #save in
delta the lowest distance just computed
    f = plt.figure()
    ax = f.add_subplot(1,1,1)
    ax.plot(rho, delta, 'o')
    return rho, delta, inds_closest_higher_rho
```

In [ ]:
```
r, d, inds = density_peaks(X, 1.0)
```

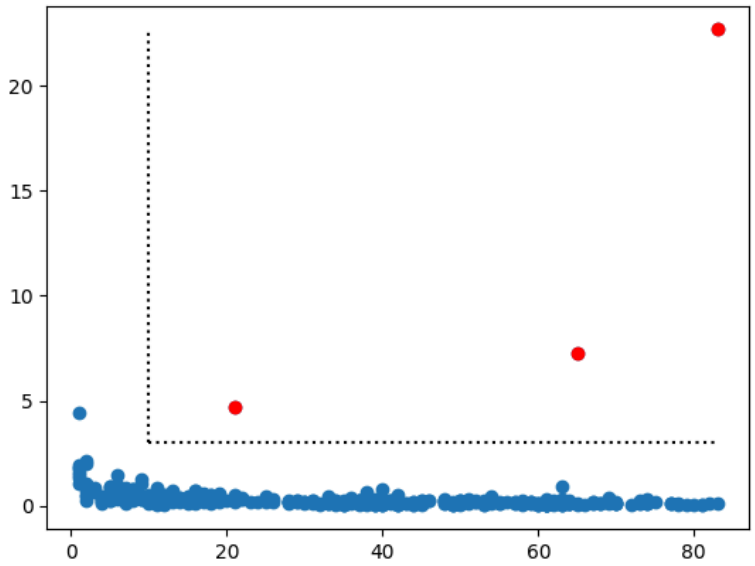
Write a function that takes as inputs the arrays produced by the previous function, and threshold values for rho, **min_rho**, and delta, **min_delta**. The function should:
* define as cluster centers all the points with rho > min_rho and delta > min_delta. I suggest to highlight the cluster centers in a plot of delta Vs rho, to check that the correct samples were selected;
* assign each point to the same cluster of its closest sample of higher density. To do this I suggest to:
* assign a cluster label to the cluster centers
* make a cycle over samples in decreasing order of density
* if the sample does not have a cluster label already assigned to it, use the cluster label of its closest sample of higher density

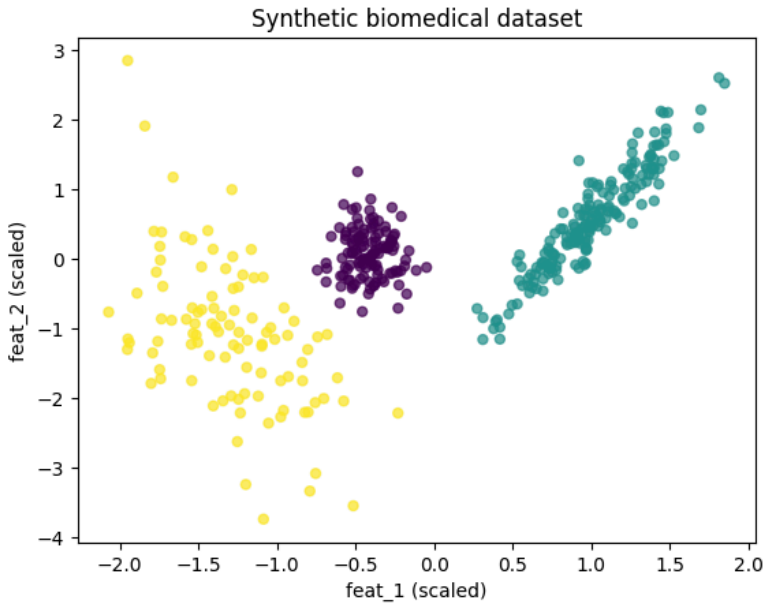
The function should return the labels of the clusters. Apply the function to the synthetic data, and plot the data using different colors for the clusters. Define the values of min_rho and min_delta based on the plot produced by the previous function.

In [ ]:
```
def clustering_dp(rho, delta, inds_closest_higher_rho, min_rho, min_delta):
#define the clustering function
    inds_centers = np.logical_and(rho > min_rho, delta > min_delta) #find the
points that are over both thresholds (outliers)
    f = plt.figure() #initialize new graph
    ax = f.add_subplot(1,1,1)
    ax.plot(rho, delta, 'o') #new scatter plot of all points in the decision graph
space
    ax.plot([min_rho, np.max(rho)], [min_delta, min_delta], 'k:') #define the
borders of the thresholds imposed
    ax.plot([min_rho, min_rho], [min_delta, np.max(delta)], 'k:')
    ax.plot(rho[inds_centers], delta[inds_centers], 'ro') #red points on the
points found to be centers
    labels = -1*np.ones(len(rho), dtype = int)
    labels[np.where(inds_centers)[0]] = np.arange(np.sum(inds_centers)) #finds the
centers and gives them a number progressively
    for ind_sample in np.argsort(rho)[::-1]: #studies all points in decreasing
density order and assign
        if labels[ind_sample] == -1: #checks if the actual point is not a cluster
center (still -1)
            labels[ind_sample] = labels[inds_closest_higher_rho[ind_sample]]
#assigns the point the same label as the closest high density center
    return labels
```

In [ ]:
```
clustering_dp(r, d, inds, 10.0, 3.0)
```

In [ ]:
```
labels = clustering_dp(r, d, inds, 10.0, 3.0) #calls the function with the density
vector, the distance vector and the labels vector
f = plt.figure()
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    s=25,
    c=labels,
    alpha=0.7
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title("Synthetic biomedical dataset")
plt.show()
```





3. Clustering of the MNIST dataset based on density peaks

The MNIST dataset includes images of hand-written digits. Each image is 8x8. The block of code below load the data, **X**, the correct labels, **y**, and it plots a random image from the dataset.

* Use the functions defined before to identify possible candidates as cluster centers.Modify the value of epsilon to get a number of isolated density peaks around 10-15
* Perform the clustering
* Compute the adjusted rank index of the clustering results
* Plots some examples of images for all the clusters identified

In [ ]:
```
images = datasets.load_digits() #loads in memory the dataset of digits
X = images['data'] #estrae features
n_images = X.shape[0] #looking at the dimension of the first digit we understand
the number of images
y = images['target'] #estrae ground truth
print('Number of images:', n_images)
X_images = X.reshape(n_images, 8, 8) #transform the flat bidimensional matrix into
a 8x8 in 3D
ind = np.random.randint(n_images) #generate a random
plt.imshow(X_images[ind], cmap='grey') #takes the 8x8 of the samples randomed
plt.title(y[ind])
```

In [ ]:
```
# this block of code imports a different dataset with images at higher resolution.
I suggest to use the previous one,
# and then maybe explore this one. In case remember that you need to adjust the
parameters of the algoriths,
# as the average distance between samples would be significanly different for
these images
from sklearn.datasets import fetch_openml
n_images = 1000 # I downsample the original dataset to this number
X,y = fetch_openml('mnist_784', return_X_y=True)
inds = np.random.choice(np.arange(len(y)).astype(int), n_images)
X = X.to_numpy()
y = y.to_numpy()
X = X[inds,:]
y = y[inds]
n_images = X.shape[0]
X_images = X.reshape(n_images, 28, 28)
X = X_images.reshape(n_images, 28 * 28)
print('Number of images:', n_images)
ind = np.random.randint(n_images)
plt.imshow(X_images[ind], cmap='grey')
plt.title(y[ind])
```

In [ ]:
```
def density_peaks(X, eps, metric = 'euclidean'):
    D = pairwise_distances(X, metric = metric) #compute the pairwise distance
matrix using the specified metric
    rho = np.sum(D < eps, axis = 1) #calculate local density by counting neighbors
within radius eps
    inds_maxmin_rho = np.argsort(rho)[::-1] #sort point indices by density in
descending order
    delta = np.zeros(len(rho)) #initialize array for minimum distance to a higher-
density point
    delta[inds_maxmin_rho[0]] = np.max(D) #assign maximum distance to the highest
density point
    inds_closest_higher_rho = np.zeros(len(rho), dtype = int) #initialize array to
store the index of the closest higher-density neighbor
    for ind in range(1, len(rho)): #loop through all remaining points from highest
to lowest density
        ind_sample =inds_maxmin_rho[ind]
        inds_higher_density = inds_maxmin_rho[:ind]
        inds_closest_higher_rho[ind_sample] =
inds_higher_density[np.argmin(D[ind_sample, inds_higher_density])]
        delta[ind_sample] = np.min(D[ind_sample, inds_higher_density])
    f = plt.figure()
    ax = f.add_subplot(1,1,1)
    ax.plot(rho, delta, 'o')
    return rho, delta, inds_closest_higher_rho
```

In [ ]:
```
r, d, inds = density_peaks(X, 25)
```

In [ ]:
```
labels = clustering_dp(r, d, inds, 15, 25) #run the density peaks clustering with
the specified thresholds
n_example_images = 8
n_clusters = np.max(labels)+1 #total number of identified clusters
for i_cluster in range(n_clusters): #iterate through each identified cluster to
visualize sample images
```

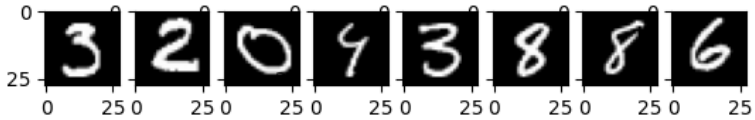
```
    i_images = np.arange(n_images)[labels == i_cluster] #get the indices of all
images belonging to the current cluster
    i_examples = np.random.choice(i_images, n_example_images) #randomly select a
subset of images from this cluster to display
    f = plt.figure()
    for j, i_image in enumerate(i_examples): #plot each selected sample image in a
side-by-side subplot layout
        ax = f.add_subplot(1, n_example_images, j+1)
        ax.imshow(X_images[i_image], cmap='grey')
ri = adjusted_rand_score(labels, y)
print('Number of clusters = ', n_clusters)
print('Adjuested rank score =', ri)
```

4. Clustering of the MNIST dataset using DBSCAN

Clusterize the same data using the DBSCAN algorithm. Set epsilon to 20 and the minimum number of samples to 10. Compute the adjusted rank index excluding the samples labelled ad noise by DBSCAN.

In [ ]:
```
db = DBSCAN(eps=20, min_samples=10).fit(X) #initialize and fit the DBSCAN
clustering model on data X
labels = db.labels_

# Number of clusters in labels, ignoring noise if present.
n_clusters_ = len(set(labels)) - (1 if -1 in labels else 0)
n_noise_ = list(labels).count(-1)

ri = adjusted_rand_score(labels, y) #evaluate overall performance against true
targets
print("Estimated number of clusters: %d" % n_clusters_)
print("Estimated number of noise points: %d" % n_noise_)
print('Adjuested rank score (with the noise "cluster") =', ri)
inds_no_noise = labels != -1 #filter out noise points to compute ARI exclusively
on valid clusters
ri = adjusted_rand_score(labels[inds_no_noise], y[inds_no_noise])
print('Adjuested rank score =', ri)

n_example_images = 8
for i_cluster in range(n_clusters_): #loop through each valid cluster to visualize
random sample images
    i_images = np.arange(n_images)[labels == i_cluster]
    i_examples = np.random.choice(i_images, n_example_images)
    f = plt.figure()
    for j, i_image in enumerate(i_examples): #display the sample images side-by-
side
        ax = f.add_subplot(1, n_example_images, j+1)
        ax.imshow(X_images[i_image], cmap='grey')

i_images = np.arange(n_images)[labels == -1]
i_examples = np.random.choice(i_images, n_example_images)
f = plt.figure()
for j, i_image in enumerate(i_examples): #display the noise sample images side-by-
side
    ax = f.add_subplot(1, n_example_images, j+1)
    ax.imshow(X_images[i_image], cmap='grey')
```

```
Estimated number of clusters: 0
Estimated number of noise points: 1000
Adjuested rank score (with the noise "cluster") = 0.0
Adjuested rank score = 1.0
```



Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Practice: Kmeans and GMM

Learning objectives

- apply K-means clustering
- apply Gaussian Mixture Models
- estimate the optimal number of clusters using:
- the elbow method
- the silhouette score
- the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (AIC)

1. Imports and setup

Run the next cell to import the libraries and initiliaze the random number generator.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score

np.random.seed(42)
plt.rcParams["figure.figsize"] = (7, 5)
```

2. Generate a synthetic dataset

Run the next cell to create a synthetic dataset with features that could resemble biomedical measurements.

In [ ]:
```
X, y_true = make_blobs(
    n_samples=450,
    centers=4,
    n_features=4,
    cluster_std=[1.1, 1.5, 1.0, 1.3],
    random_state=42
)

feature_names = [
    "inflammatory_marker",
    "hrv_index",
    "glucose_marker",
    "oxygenation_feature"
]

df = pd.DataFrame(X, columns=feature_names)
df["true_group"] = y_true
df.head()
```

2bis. A more complicated dataset

After you finished the exercise with the first dataset, create this alternative dataset and repeat everything.

Note the following differences:
* Does the Elbow method works as well as before ?
* What about the silhoutte score ?
* Is the clustering of K-means as good as before ? Why ?
* Is GMM clustering better ? Why ?

In [ ]:
```
# Cluster 1
mean1 = [1, 6]
cov1 = [[0.5, 0.0],
        [0.0, 0.5]]
X1 = np.random.multivariate_normal(mean1, cov1, 120)

# Cluster 2
mean2 = [9, 7]
cov2 = [[2.5, 1.8],
        [1.8, 1.5]]
X2 = np.random.multivariate_normal(mean2, cov2, 180)

# Cluster 3
mean3 = [-4, 4]
cov3 = [[4.2, -2.4],
        [-2.4, 4.0]]
X3 = np.random.multivariate_normal(mean3, cov3, 100)
```

```
X = np.vstack([X1, X2, X3])
y_true = np.array([0]*len(X1) + [1]*len(X2) + [2]*len(X3))
```

3. Explore and standardize the data

Clustering methods are sensitive to the scale of the variables.
For this reason, it is common to standardize the features before fitting the models.
Standardize each feature to have average equal to zero and standard deviation equal to one.

In [ ]:
```
X_features = df[feature_names].copy() #create a shallow copy of the selected
feature columns from the DataFrame

scaler = StandardScaler() #initialize the standard scaler to normalize features
X_scaled = scaler.fit_transform(X_features) #fit the scaler to the data and
transform it into standardized values

pd.DataFrame(X_scaled, columns=feature_names).describe().loc[["mean", "std"]]
#convert the scaled array back to a DataFrame and display only mean and std rows
```

Use this for dataset 2b

In [ ]:
```
feature_names = ['feat_1', 'feat_2']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pd.DataFrame(X_scaled, columns=feature_names).describe().loc[["mean", "std"]]
```

Quick visualization in 2D

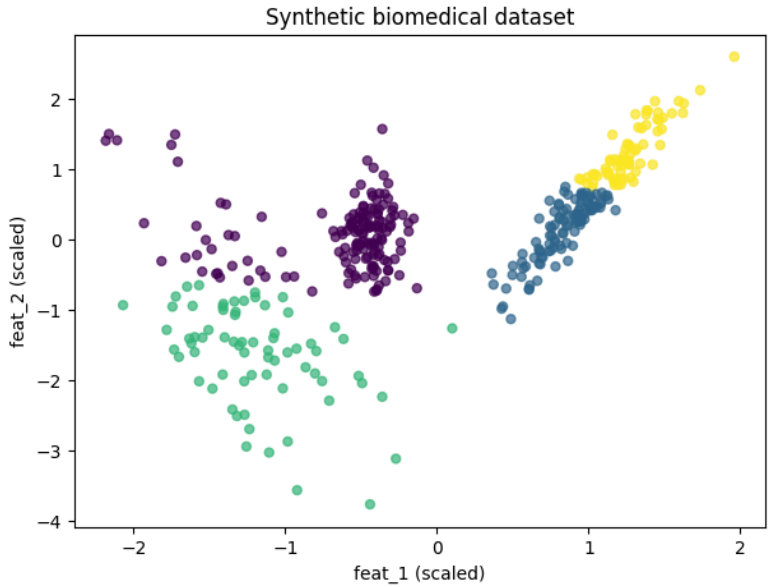
Make a scatter plot of the samples in 2D using the first two features. This is intended only for a visualization of the dataset. Clustering will be performed considering all the features.

In [ ]:
```
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    s=25,
    alpha=0.7
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title("Synthetic biomedical dataset")
plt.show()
```

4. K-means clustering

Run K-Means clustering with an arbitrary value for k.
* Use the KMeans class of scikitlearn. Check the parameters: n_clusters, random_state, n_init
* Use the method fit_predict to calculate the clusters
* Make a scatter plot in 2D with points coloured using the cluster index

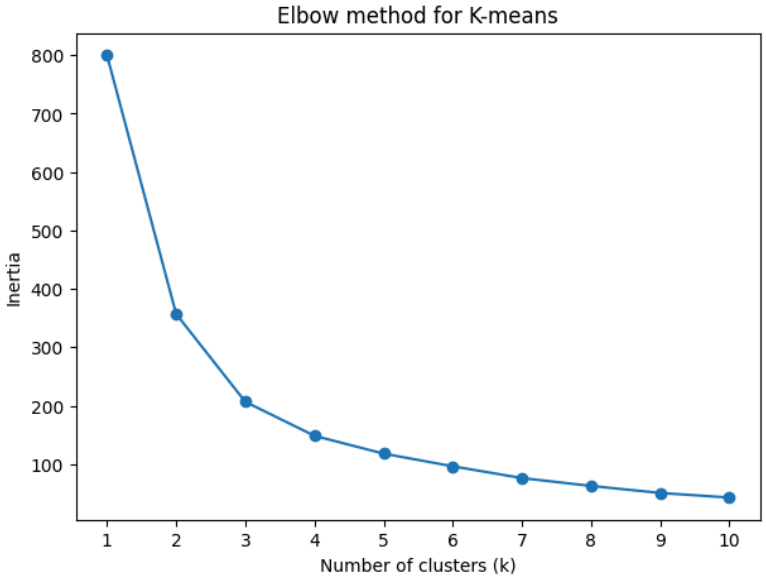
In [ ]:
```
k = 4
km = KMeans(n_clusters=k, random_state=42, n_init=20) #initialize KMeans
yc = km.fit_predict(X_scaled) #fit the model to the standardized features and
predict cluster assignments for each point
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=yc,
    s=25,
    alpha=0.7,
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title("Synthetic biomedical dataset")
plt.show()
```



5. Choosing the number of clusters for K-means: elbow method

The elbow method evaluates the inertia (within-cluster sum of squares) for different values of `k`. The optimal number of cluster is the one where the decrease in inertia starts to slow down.
* Compute the inertia for different value of k (see the inertia_ value of the KMeans class)
* Make a plot of the inertia as a function of k
* Select the optimal number of clusters

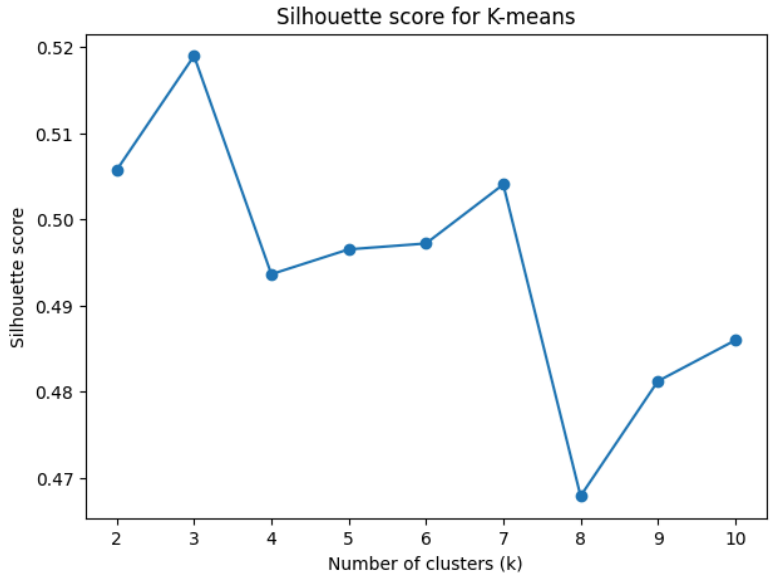
In [ ]:
```
k_values = range(1, 11) #define the range of cluster numbers to evaluate
inertias = [] #within-cluster sum of squares

for k in k_values: #loop through each k value to compute and collect the model
inertia
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(k_values, inertias, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow method for K-means")
plt.xticks(list(k_values))
plt.show()
```



6. Choosing the number of clusters for K-means: silhouette score

The silhouette score measures how well separated the clusters are. For each sample, it compares the average distance to points in the same cluster with the average distance to points in the nearest different cluster.
* Compute the silhoutte_score for the same values of k tested before with the elbow method using the silhouette_score function from sklearn
* Choose the optimal number of clusters

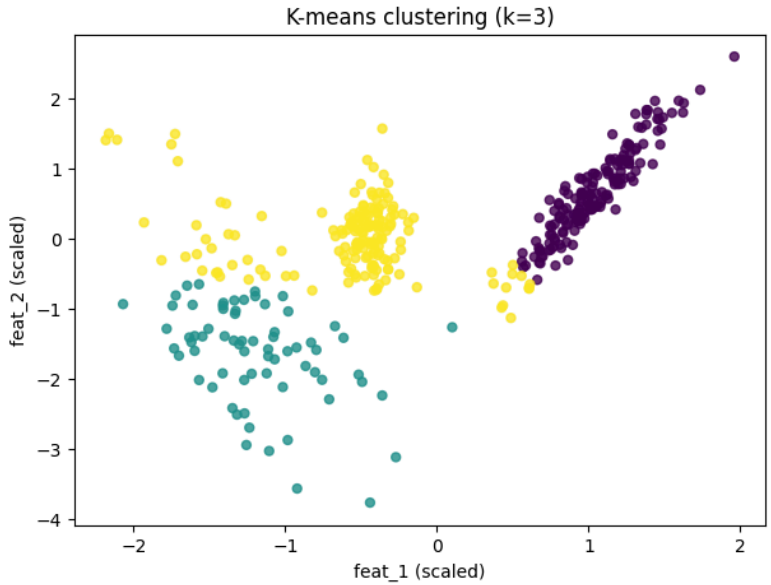
In [ ]:
```
k_values = range(2, 11)
sil_scores = []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = km.fit_predict(X_scaled) #fit the model and predict cluster
assignments for all samples
    score = silhouette_score(X_scaled, labels) #calculate the mean silhouette
score across all samples for this clustering
    sil_scores.append(score)

plt.plot(list(k_values), sil_scores, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette score")
plt.title("Silhouette score for K-means")
plt.xticks(list(k_values))
plt.show()
```



7. Fit the final K-means model

Fit the model using the optimal number of clusters defined above. Visualize the K-means results on two dimensions using different marker colors for each cluster.

In [ ]:
```
best_k = 3

kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=20)
kmeans_labels = kmeans.fit_predict(X_scaled)

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=kmeans_labels,
    s=25,
    alpha=0.8
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title(f"K-means clustering (k={best_k})")
plt.show()
```



8. Gaussian Mixture Models (GMM)

A Gaussian Mixture Model assumes that the data come from a mixture of several Gaussian distributions.

Compared with K-means, GMM can model clusters that are:

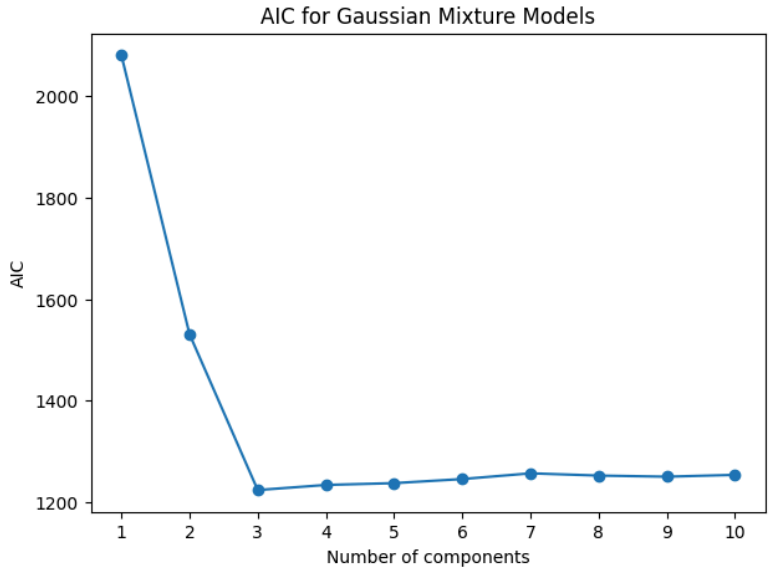
- elliptical
- overlapping
- different in size and orientation

9. Choosing the number of clusters/components for GMM: AIC/BIC

For Gaussian Mixture Models, a common selection criterion is the Akaike Information Criterion (AIC) or the Bayesian Information Criterion (AIC). Unlike inertia, AIC/BIC explicitly penalize overly complex models.

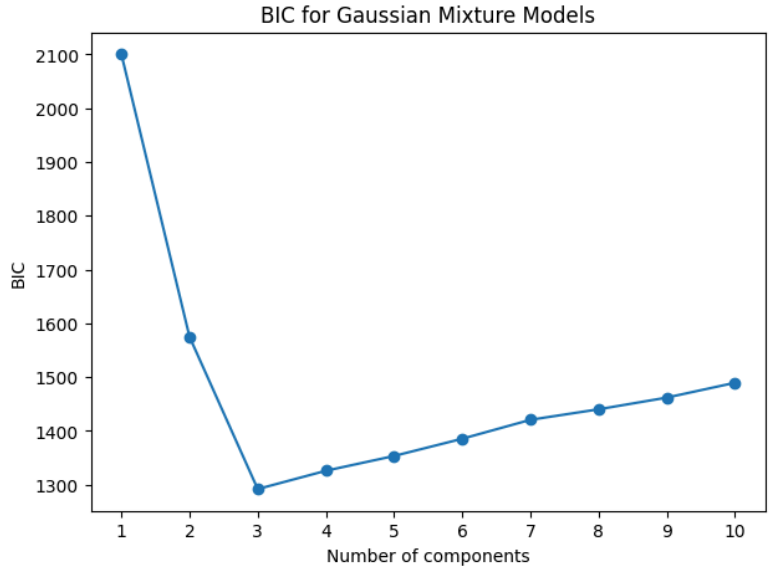
* Compute the AIC value for GMM with different number of components
* Select the optimal number of components
* Repeat the previous points using BIC

In [ ]:
```
n_components_range = range(1, 11) #define the range of mixture components
aic_values = []

for n in n_components_range:
    gmm = GaussianMixture(n_components=n, covariance_type="full", random_state=42)
#initialize Gaussian Mixture Model
    gmm.fit(X_scaled) #fit the GMM on the standardized features
    aic_values.append(gmm.aic(X_scaled)) #compute and append the AIC score for the
fitted model configuration

plt.plot(list(n_components_range), aic_values, marker="o")
plt.xlabel("Number of components")
plt.ylabel("AIC")
plt.title("AIC for Gaussian Mixture Models")
plt.xticks(list(n_components_range))
plt.show()

pd.DataFrame({"n_components": list(n_components_range), "AIC": aic_values})
```



	n_components	AIC
0	1	2080.436225
1	2	1530.836739
2	3	1224.070578
3	4	1234.136528
4	5	1237.680362
5	6	1245.744327
6	7	1257.002356
7	8	1252.597681
8	9	1250.380696
9	10	1253.988762

In [ ]:
```
n_components_range = range(1, 11)
bic_values = []

for n in n_components_range:
    gmm = GaussianMixture(n_components=n, covariance_type="full", random_state=42)
    gmm.fit(X_scaled)
    bic_values.append(gmm.bic(X_scaled)) #compute and append the BIC score for the
fitted model configuration

plt.plot(list(n_components_range), bic_values, marker="o")
plt.xlabel("Number of components")
plt.ylabel("BIC")
plt.title("BIC for Gaussian Mixture Models")
plt.xticks(list(n_components_range))
plt.show()

pd.DataFrame({"n_components": list(n_components_range), "BIC": bic_values})
```



	n_components	BIC
0	1	2100.393548
1	2	1574.742849
2	3	1291.925475
3	4	1325.940212
4	5	1353.432834
5	6	1385.445587
6	7	1420.652403
7	8	1440.196514
8	9	1461.928317
9	10	1489.485171

10. Fit the final GMM model

Fit the final GMM model with the optimal number of clusters defined above. Visualize the results on two dimensions using a different marker color for each cluster.

In [ ]:
```
best_n_components = 3 #set the optimal number of Gaussian components determined
from previous metrics

gmm = GaussianMixture( #initialize the Gaussian Mixture Model with the chosen
components
    n_components=best_n_components,
    covariance_type="full",
    random_state=42
)

gmm_labels = gmm.fit_predict(X_scaled) #fit the GMM and assign each sample to the
component with the highest posterior probability
gmm_probabilities = gmm.predict_proba(X_scaled) #compute the soft assignment
probabilities

plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=gmm_labels,
    s=25,
    alpha=0.8
)
plt.xlabel(feature_names[0] + " (scaled)")
plt.ylabel(feature_names[1] + " (scaled)")
plt.title(f"GMM clustering (components={best_n_components})")
plt.show()
```

11. Compare K-means and GMM

Compare the silhouette scores of the selected models respectivelly for K-means and GMM.

In [ ]:
```
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels) #calculate the mean
silhouette score for the K-means clustering configuration
gmm_silhouette = silhouette_score(X_scaled, gmm_labels) #calculate the mean
silhouette score for the Gaussian Mixture Model configuration

comparison = pd.DataFrame({ #build a summary DataFrame to compare performance
metrics
    "method": ["K-means", "GMM"],
    "n_clusters_or_components": [best_k, best_n_components],
    "silhouette_score": [kmeans_silhouette, gmm_silhouette]
})

comparison
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Practice: K-means and Hierarchical Clustering

Learning objectives

By the end of this practice you will be able to:

- apply K-means and hierarchical clustering
- compare clustering results using visual inspection and quantitative metrics
- discuss when centroid-based and hierarchical approaches are more appropriate

1. Imports

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA

from scipy.cluster.hierarchy import dendrogram, linkage

np.random.seed(42)
```

2. Create a synthetic dataset

The dataset below includes four subpopulations that are not well represented by spherical clusters.

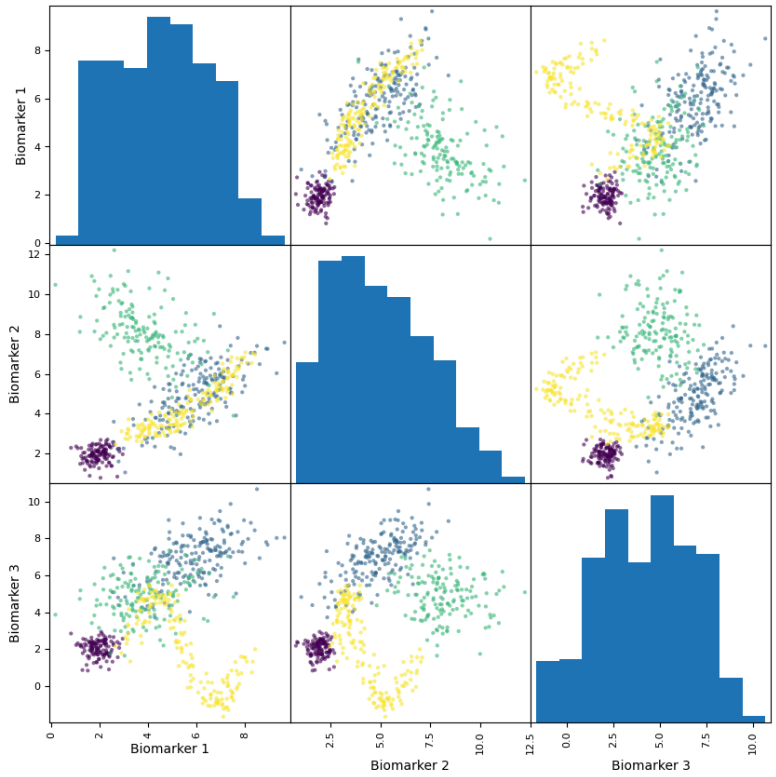
In [ ]:
```
# ---- Cluster 1 (compact) ----
mean1 = [2, 2, 2]
cov1 = [[0.2, 0.05, 0.0],
        [0.05, 0.2, 0.05],
        [0.0, 0.05, 0.2]]
X1 = np.random.multivariate_normal(mean1, cov1, 120)

# ---- Cluster 2 (elongated) ----
mean2 = [6, 5, 7]
cov2 = [[2.0, 1.5, 1.0],
        [1.5, 2.0, 1.2],
        [1.0, 1.2, 1.5]]
X2 = np.random.multivariate_normal(mean2, cov2, 180)

# ---- Cluster 3 (overlapping, high variance) ----
mean3 = [4, 8, 5]
cov3 = [[1.5, -0.8, 0.3],
        [-0.8, 2.0, -0.5],
        [0.3, -0.5, 1.5]]
X3 = np.random.multivariate_normal(mean3, cov3, 150)

# ---- Cluster 4 (curved trajectory) ----
t = np.linspace(0, 1, 160)
x = 3 + 5*t
y = 3 + 4*(t**2)
z = 2 + 3*np.sin(2*np.pi*t)
curve = np.vstack([x, y, z]).T
noise = np.random.normal(scale=0.3, size=curve.shape)
X4 = curve + noise

# ---- Combine dataset ----
X = np.vstack([X1, X2, X3, X4])
y_true = np.array(
    [0]*len(X1) +
    [1]*len(X2) +
    [2]*len(X3) +
    [3]*len(X4)
)

feature_names = [
    "Biomarker 1",
    "Biomarker 2",
    "Biomarker 3",
]

df = pd.DataFrame(X, columns=feature_names)
df["true_group"] = y_true
df.head()
```

3. Quick exploration

Make 2D plots of all the possible projections. The 4 clusters are not perfecly separated by any 2D projections. Choose the projection that better separates the clusters and use that one for the rest of the exercise (but remember that: 1) clustering works with all the features; 2) in a real case you don't know the "true" clusters).

In [ ]:
```
pd.plotting.scatter_matrix(
    df[feature_names],
    figsize=(10, 10),
    diagonal="hist",
    alpha=0.6,
    c=y_true
)
plt.suptitle("Scatter matrix of synthetic biomedical features", y=1.02)
plt.show()
```

Scatter matrix of synthetic biomedical features



4. Standardize the features

Normalize the features to zero mean and unitary standard deviation

In [ ]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_names])
```

6. K-means clustering

Try to define the optimal number of clusters using the elbow method and the silhoutte score.

In [ ]:
```
k_values = range(2, 8)
kmeans_inertia = []
kmeans_silhouette = []

for k in k_values:
    model = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = model.fit_predict(X_scaled)
    kmeans_inertia.append(model.inertia_)
    kmeans_silhouette.append(silhouette_score(X_scaled, labels))

fig, ax = plt.subplots(1, 2, figsize=(12, 4))

ax[0].plot(list(k_values), kmeans_inertia, marker="o")
ax[0].set_xlabel("Number of clusters k")
ax[0].set_ylabel("Inertia")
ax[0].set_title("K-means elbow curve")

ax[1].plot(list(k_values), kmeans_silhouette, marker="o")
ax[1].set_xlabel("Number of clusters k")
ax[1].set_ylabel("Silhouette score")
ax[1].set_title("K-means silhouette analysis")

plt.tight_layout()
plt.show()
```

* Perform clustering with Kmeans using 4 clusters (even if that's probably not what you should do based on the plots before)
* Create a scatter plot in 2D over the features selected before with samples coloured as the Kmeans labels

In [ ]:
```
kmeans = KMeans(n_clusters=4, n_init=20, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)
```

In [ ]:
```
plt.figure(figsize=(7, 6))
plt.scatter(X_scaled[:, 1], X_scaled[:, 2], c=kmeans_labels, s=25, alpha=0.7)
plt.xlabel("Biomarker 2 ")
plt.ylabel("Biomarker 3")
plt.title("K-means clusters")
plt.show()
```

7. Hierarchical clustering

* Perform hierarchical clustering using the class AgglomerativeClustering
* Test the different linkage methods: ward, complete, average, and single
* For each method compute the silhoutte score

In [ ]:
```
linkages = ["ward", "complete", "average", "single"] #define the different linkage
criteria to evaluate for hierarchical clustering
hier_results = []

for linkage_name in linkages: #loop through each linkage method to perform
clustering and compute its performance
    model = AgglomerativeClustering(n_clusters=4, linkage=linkage_name)
#initialize the Agglomerative Clustering model with 4 target clusters and current
linkage
    labels = model.fit_predict(X_scaled) #fit the hierarchical model and predict
cluster assignments for all samples
    sil = silhouette_score(X_scaled, labels) #calculate the mean silhouette score
across all samples for this configuration
    hier_results.append((linkage_name, sil)) #append the linkage name and its
corresponding score to the collection list

results_df = pd.DataFrame(
    hier_results,
    columns=["linkage", "silhouette_score", ]
).sort_values(["silhouette_score"], ascending=False)
results_df
```

* Perform clustering using the best linkage method identified

In [ ]:
```
best_linkage = results_df.iloc[0]["linkage"] #extract the best performing linkage
method from the first row of the sorted results DataFrame
hierarchical = AgglomerativeClustering(n_clusters=4, linkage=best_linkage)
#initialize the Agglomerative Clustering model with 4 target clusters and the
optimal linkage
hier_labels = hierarchical.fit_predict(X_scaled) #fit the hierarchical model and
predict cluster assignments for all samples

print(f"Best linkage according to ARI (teaching only): {best_linkage}")
print(f"Hierarchical silhouette score: {silhouette_score(X_scaled,
hier_labels):.3f}")
```

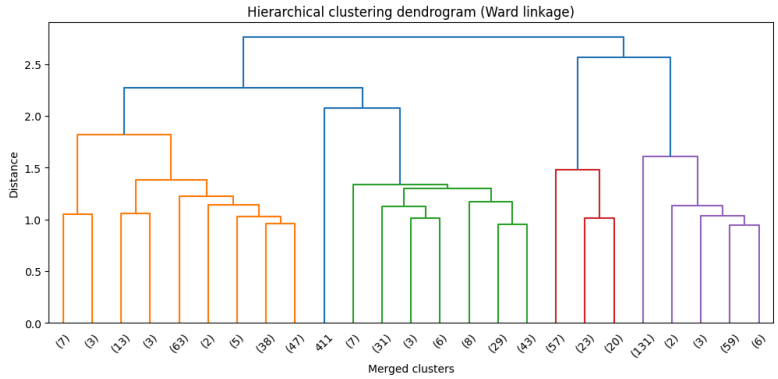
```
Best linkage according to ARI (teaching only): average
Hierarchical silhouette score: 0.500
```

In [ ]:
```
plt.figure(figsize=(7, 6))
plt.scatter(X_scaled[:, 1], X_scaled[:, 2], c=hier_labels, s=25, alpha=0.7)
plt.xlabel("Biomarker 2 ")
plt.ylabel("Biomarker 3")
plt.title("K-means clusters")
plt.show()
```

8. Dendrogram

Use the methods linkage and dendrogram to plot the hierarchical tree connecting the samples in the dataset.

In [ ]:
```
Z = linkage(X_scaled, method=best_linkage) #compute the hierarchical clustering
linkage matrix using the best performing linkage method

plt.figure(figsize=(12, 5))
dendrogram(Z, truncate_mode="lastp", p=25, leaf_rotation=45, leaf_font_size=10)
#plot the dendrogram, truncating the tree to show only the last 25 merged clusters
for clarity
plt.title("Hierarchical clustering dendrogram (Ward linkage)")
plt.xlabel("Merged clusters")
plt.ylabel("Distance")
plt.show()
```



9. Compare K-means and hierarchical clustering

* Compare the two methods with respect to silhoutte score and adjusted rank index

In [ ]:
```
comparison = pd.DataFrame({
    "Method": ["K-means", f"Hierarchical ({best_linkage})"],
    "Silhouette score": [
        silhouette_score(X_scaled, kmeans_labels),
        silhouette_score(X_scaled, hier_labels)
```

```
    ],
    "Adjusted Rand index": [
        adjusted_rand_score(y_true, kmeans_labels),
        adjusted_rand_score(y_true, hier_labels)
    ]
})

comparison
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Practice: Dimensionality reduction algorithms

Learning objectives

* apply PCA to a reduce the dimensionality of a high-dimensional dataset
* Select the optimal number of principal components
* repeat the exercise with IsoMap/tSNA/UMAP, and analyze how the projection depends on the model parameters

0. Imports and setup

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.manifold import Isomap
from sklearn.manifold import TSNE

np.random.seed(42)
```

1. Import and refine data

The array X includes the expression value of 2000 genes for 2700 cells obtained by single-cell RNAseq. The kind of the cell, as defined by known genetic markers, is reported in the array y.

In [ ]:
```
data = np.load("/content/sample_data/data_pbmc.npz")
X = data["X"]
y = data["y"]
```

Remove from X and y all the cells that are labelled as 'Unknown'.
* How many cells are left ?
* Create an array, y_int, with the cell types converted into integer values, and an array, labels, with the list of cell types (Suggestion: use the function np.unique)

In [ ]:
```
inds = y != 'Unknown'
X = X[inds,:] #filter out rows with 'Unknown' labels from the feature matrix X
y = y[inds]
labels, y_int = np.unique(y, return_inverse=True) #extract unique class names and
map text labels to integer codes
print('Number of samples:', X.shape[0])
print('Possible cell types:', labels)
```

```
Number of samples: 2597
Possible cell types: ['B cells' 'Dendritic cells' 'Megakaryocytes'
'Monocytes' 'NK cells'
 'T cells']
```

2. Principal Component Analysis

* Project the data onto the first 50 principal components
* Plot the cumulative variance explained by the PCs
* Plot the data over the first 2 PCs using different markers (or colors) for the various cell kinds
* Compute the silhoutte score using the first 2 PCs and the labels in y_int. How do you interpret the silhoutte score in this case ?

In [ ]:
```
n_pc = 50
pca = PCA(n_components = n_pc)
pca.fit(X) #fit the PCA model to calculate the orthogonal variance axes of data X
Xpca = pca.transform(X) #project the high-dimensional data into the lower-
dimensional PCA subspace
```

In [ ]:
```
f = plt.figure()
ax = f.add_subplot(1,1,1)
ax.plot(np.arange(n_pc), np.cumsum(pca.explained_variance_ratio_), '-ok') #plot
the cumulative sum of explained variance ratio against the component indices
```

In [ ]:
```
f = plt.figure()
ax = f.add_subplot(1,1,1)
for i_label, label in enumerate(labels): #loop through unique labels to plot each
class with a different color
    inds = y_int == i_label #create a boolean mask for samples belonging to the
current class index
    ax.plot(Xpca[inds,0], Xpca[inds,1], '+', label = label)
plt.legend()
```

In [ ]:
```
sc = silhouette_score(Xpca[:,:2], y_int)
print('Silhoutte score with 2 PCs:', sc)
```

```
Silhoutte score with 2 PCs: 0.2711054691710063
```

3. Dimensionality reduction with IsoMap

* Use the IsoMap method to project the data over a 2-dimensional space starting from the original data X
* Change the value of the number of neighbors in the range from 5 to 20, and select the value that optimize the silhoutte score. Can you get a decent separation of the different cell types ?
* Repeat the previous points using as input the projection of the data over the first 50 principal components
* Select the number of neighbors that provides the highest silhoutte score
* Plot the data in the projected space using different colors for the various cell types

In [ ]:
```
ns_neighbors = np.arange(5,21,2).astype(int) #define an array of neighbor sizes to
evaluate
scores = np.zeros(len(ns_neighbors)) #initialize an array of zeros to store the
silhouette score
for i, n_neighbors in enumerate(ns_neighbors): #loop through each neighbor value
to fit Isomap
    iso = Isomap(n_components=2, n_neighbors=n_neighbors)
    Xiso = iso.fit_transform(X) #fit the manifold model and transform the high-
dimensional data X into the 2D subspace
    scores[i] = silhouette_score(Xiso, y_int)
    print('Silhoutte score for IsoMap with n_neighbors {}: {}'.format(n_neighbors,
scores[i]))
```

In [ ]:
```
ns_neighbors = np.arange(5,21,2).astype(int)
scores = np.zeros(len(ns_neighbors))
for i, n_neighbors in enumerate(ns_neighbors):
    iso = Isomap(n_components=2, n_neighbors=n_neighbors)
    Xiso = iso.fit_transform(Xpca) #here we pass the pca data
    scores[i] = silhouette_score(Xiso, y_int)
    print('Silhoutte score for IsoMap with n_neighbors {}: {}'.format(n_neighbors,
scores[i]))
```

In [ ]:
```
n_neighbor = ns_neighbors[np.argmax(scores)] #identify the optimal number of
neighbors that yielded the highest silhouette score
print('Using {} neighbors'.format(n_neighbor))
iso = Isomap(n_components=2, n_neighbors=n_neighbor) #initialize Isomap with the
optimal neighbor count
Xiso = iso.fit_transform(Xpca) #fit and transform the PCA-reduced data into the
optimized 2D Isomap embedding
f = plt.figure()
ax = f.add_subplot(1,1,1)
for i_label, label in enumerate(labels): #loop through unique classes to scatter
plot the embedding points by group
    inds = y_int == i_label #create a boolean mask for samples belonging to the
current class
    ax.plot(Xiso[inds,0], Xiso[inds,1], '+', label = label)
plt.legend()
```

4. Dimensionality reduction with tSNE

* Use the TSNE method to project the data over a 2-dimensional space starting from the projection of the data over the first 50 principal components
* Test different values of perplexity and observe how the projection changes

In [ ]:
```
perplexity = 10
tsne = TSNE(n_components=2, perplexity=perplexity, random_state=42) #initialize
the t-SNE model to project the data into a 2D space with a reproducible seed
Xtsne = tsne.fit_transform(Xpca) #fit the manifold model and transform the PCA-
reduced data into the 2D t-SNE embedding
f = plt.figure()
ax = f.add_subplot(1,1,1)
for i_label, label in enumerate(labels): #loop through unique classes to scatter
plot the embedding points by group
    inds = y_int == i_label
    ax.plot(Xtsne[inds,0], Xtsne[inds,1], '+', label = label)
plt.legend()
```

4. Dimensionality reduction with UMAP

* Create a conda environment by cloning the base environment (conda create --name NAME_NEW_ENV --clone base)
* Activate the new environment (conda activate NAME_NEW_ENV)
* Install the package umap-learn from the channel conda-forge (conda install -c conda-forge umap-learn
* Import the umap library
* Use the umap method to project the data over a 2-dimensional space starting from the projection of the data over the first 50 principal components
* Test different values of the number of neighbors and observe how the projection changes

In [ ]:
```
import umap
```

In [ ]:
```
n_neighbors = 4 #same
min_dist = 0.1 # This value was missing and caused the NameError
uma = umap.UMAP(n_components=2, n_neighbors=n_neighbors, min_dist=min_dist,
random_state=42, n_jobs = 1)
Xuma = uma.fit_transform(Xpca)
f = plt.figure()
ax = f.add_subplot(1,1,1)
for i_label, label in enumerate(labels):
    inds = y_int == i_label
```

```
    ax.plot(Xuma[inds,0], Xuma[inds,1], '+', label = label)
plt.legend()
```

5. Clustering in the low dimensional space

* Clusterize the data in the low-dimensional space obtained by UMAP with DBSCAN. Optimize the parameters of DBSCAN by looking at the clusterized data in the low-dimensional space
* Use the function imshow to plot an heatmap of gene expression values. Restrict the scale of the heatmap in the range from -0.1 to 0.1
* Order the cells based on the DBSCAN clustering results

In [ ]:
```
from sklearn.cluster import DBSCAN
model = DBSCAN(eps=0.5, min_samples=5).fit(Xuma) #initialize DBSCAN with a maximum
neighborhood radius of 0.5 and a minimum of 5 samples per core point
f = plt.figure()
ax = f.add_subplot(1,1,1)
unique_labels = np.unique(model.labels_) #added(?)
n_clusters = len(unique_labels[unique_labels != -1]) #added(?)
for i_label in range(n_clusters): #loop through the expected number of clusters to
plot each detected group sequentially
    inds = model.labels_ == i_label
    ax.plot(Xuma[inds,0], Xuma[inds,1], '+', label = i_label)
plt.legend()
inds = model.labels_ == -1 #create a boolean mask specifically to identify
unassigned noise points
ax.plot(Xuma[inds,0], Xuma[inds,1], 'xk', label = -1)
```

In [ ]:
```
inds = np.argsort(model.labels_) #get the indices that sort the DBSCAN cluster
labels in ascending order to group similar points together
plt.imshow(X[inds], cmap="viridis", vmin = -0.1, vmax = 0.1)
plt.colorbar()
plt.show()
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Logistic Regression Regularization: Comparison of L2, L1, and Firth's Logistic Regression

You will analyze a simulated clinical-biomarker dataset designed to contain three common problems in biomedical data:

1. limited sample size,
2. correlated predictors,
3. quasi-complete separation.

You will compare:

- unregularized logistic regression,
- L2-regularized logistic regression,
- L1-regularized logistic regression,
- logistic regression with elastic-net regularization,
- Firth's bias-reduced logistic regression.

The emphasis is not only predictive performance, but also coefficient stability, feature selection, and behavior under separation.

Biological scenario

Suppose you are developing a diagnostic model for detecting a rare inflammatory cardiac condition. The available predictors include demographic variables, standard clinical measurements, blood biomarkers, and imaging-derived variables. The disease is rare, and some imaging markers are highly predictive, producing quasi-complete separation in small samples.

The outcome is binary:

$$
Y_i =
\begin{cases}
1, & \text{if disease is present},\\
0, & \text{if disease is absent}.
\end{cases}
$$

The goal is to compare how different logistic regression approaches behave under these conditions.

1. Import libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_predict
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, brier_score_loss, accuracy_score,
confusion_matrix
from sklearn.calibration import calibration_curve
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

np.random.seed(42)
```

2. Generate a synthetic dataset

The dataset has:

- 18 predictors,
- correlated biomarker groups,
- 4 truly disease-associated predictors,
- class imbalance,
- one predictor designed to create quasi-complete separation.

The data are simulated so that the true data-generating process is known. This lets you assess whether each method recovers the true signal or overreacts to artifacts.

In [ ]:
```
def simulate_biomedical_data(n=200, random_state=42):
    # Initialize the NumPy random number generator for reproducibility
    rng = np.random.default_rng(random_state)

    # Generate latent standard normal factors to introduce multi-variable
correlation blocks
```

```
    z1 = rng.normal(size=n)
    z2 = rng.normal(size=n)
    z3 = rng.normal(size=n)

    # Generate independent baseline clinical covariates (demographics and vitals)
    age = 55 + 12 * rng.normal(size=n)
    bmi = 27 + 4 * rng.normal(size=n)
    sbp = 120 + 15 * rng.normal(size=n)

    # Generate an inflammatory biomarker block sharing common variance via z1
    crp = 0.8 * z1 + 0.4 * rng.normal(size=n)
    il6 = 0.75 * z1 + 0.5 * rng.normal(size=n)
    fibrinogen = 0.65 * z1 + 0.6 * rng.normal(size=n)

    # Generate a structural cardiac imaging block sharing common variance via z2
    wall_thickness = 0.9 * z2 + 0.4 * rng.normal(size=n)
    edema_score = 0.8 * z2 + 0.5 * rng.normal(size=n)
    strain_abnormality = 0.7 * z2 + 0.6 * rng.normal(size=n)

    # Generate a metabolic profile block sharing common variance via z3
    glucose = 0.8 * z3 + 0.6 * rng.normal(size=n)
    insulin = 0.75 * z3 + 0.6 * rng.normal(size=n)
    triglycerides = 0.65 * z3 + 0.7 * rng.normal(size=n)

    # Generate pure white noise features to test feature selection algorithms
    noise = rng.normal(size=(n, 6))

    # Assemble all generated continuous features into a unified DataFrame
    X = pd.DataFrame({
        "age": age,
        "bmi": bmi,
        "systolic_bp": sbp,
        "crp": crp,
        "il6": il6,
        "fibrinogen": fibrinogen,
        "wall_thickness": wall_thickness,
        "edema_score": edema_score,
        "strain_abnormality": strain_abnormality,
        "glucose": glucose,
        "insulin": insulin,
        "triglycerides": triglycerides,
        "noise_1": noise[:, 0],
        "noise_2": noise[:, 1],
        "noise_3": noise[:, 2],
        "noise_4": noise[:, 3],
        "noise_5": noise[:, 4],
        "noise_6": noise[:, 5],
    })

    # Standardize data to zero mean and unit variance for stable parameter scaling
    Xs = (X - X.mean()) / X.std(ddof=0)

    # Define the true sparse linear predictor using a subset of continuous
features
    eta = (
        -4.0
        + 0.55 * Xs["age"]
        + 0.95 * Xs["crp"]
        + 1.15 * Xs["edema_score"]
        + 0.65 * Xs["glucose"]
    )

    # Map the linear predictor to a probability scale using the standard logistic
function
    p = 1 / (1 + np.exp(-eta))

    # Sample the binary outcome vector from the computed Bernoulli probabilities
    y = rng.binomial(1, p)

    # Synthesize a binary imaging flag engineered to exhibit quasi-separation
relative to y
    sep = np.where(y == 1, rng.binomial(1, 0.88, n), rng.binomial(1, 0.04, n))
    X["late_gadolinium_flag"] = sep

    return X, pd.Series(y, name="disease")

# Execute the simulator function to obtain features and targets
X, y = simulate_biomedical_data()

# Store feature names for indexing downstream models
features = X.columns

# Display the head of the generated feature set
X.head()
```

3. Implemention of Firth's logistic regression

Firth's logistic regression maximizes the penalized likelihood

$$
\ell_F(\beta) = \ell(\beta) + \frac{1}{2}\log |I(\beta)|,
$$

where I(beta) is the Fisher information matrix. This correction reduces first-order small-sample bias and gives finite estimates in many cases of complete or quasi-complete separation.

The implementation below uses the adjusted-score Newton algorithm.

In [ ]:
```
class FirthLogisticRegression(BaseEstimator, ClassifierMixin):
    """Small educational implementation of Firth's logistic regression.

    This implementation is intended for teaching and moderate-size datasets.
    It supports binary outcomes and dense numeric predictors.
    """

    def __init__(self, max_iter=100, tol=1e-7, step_halving=25): #initialize
optimization hyper-parameters for convergence and line search
```

```
        self.max_iter = max_iter
        self.tol = tol
        self.step_halving = step_halving

    @staticmethod
    def _sigmoid(z): #clip input values to prevent underflow/overflow errors in
exponential execution
        z = np.clip(z, -35, 35)
        return 1 / (1 + np.exp(-z))

    def _penalized_loglik(self, X_design, y, beta): #calculate linear predictors
and map them to conditional probabilities
        eta = X_design @ beta
        mu = self._sigmoid(eta)
        eps = 1e-12
        loglik = np.sum(y * np.log(mu + eps) + (1 - y) * np.log(1 - mu + eps))
#compute the baseline unpenalized Bernoulli log-likelihood
        w = mu * (1 - mu) #construct the diagonal weight matrix entries from the
variance profile
        I = X_design.T @ (w[:, None] * X_design) #calculate the Fisher Information
matrix for the current sample coordinates
        sign, logdet = np.linalg.slogdet(I + 1e-9 * np.eye(I.shape[0])) #compute
the log-determinant of the Information matrix using a small ridge penalty for
numerical stability
        if sign <= 0:
            return -np.inf #return Jeffreys-prior penalized log-likelihood object
        return loglik + 0.5 * logdet

    def fit(self, X, y): #enforce floating-point NumPy matrix representations for
inputs
        X = np.asarray(X, dtype=float)
        y = np.asarray(y, dtype=float)
        n, p = X.shape
        X_design = np.column_stack([np.ones(n), X]) #horizontal stack to
incorporate a column of ones for the intercept term
        beta = np.zeros(p + 1) #initialize the coefficient vector

        for iteration in range(self.max_iter): #execute Modified Newton-Raphson
optimization loop
            eta = X_design @ beta #compute current probabilities and variance
weight factors
            mu = self._sigmoid(eta)
            w = np.clip(mu * (1 - mu), 1e-9, None)

            I = X_design.T @ (w[:, None] * X_design) #reconstruct Fisher
Information matrix and compute its Moore-Penrose pseudo-inverse
            I_inv = np.linalg.pinv(I)

            # Diagonal of the hat matrix H = W^(1/2) X I^-1 X' W^(1/2)
            h = np.sum((X_design @ I_inv) * X_design, axis=1) * w

            #compute the Firth-adjusted score vector incorporating the leverage
weights
            adjusted_score = X_design.T @ (y - mu + h * (0.5 - mu))
            delta = I_inv @ adjusted_score

            old_obj = self._penalized_loglik(X_design, y, beta) #store the current
objective value to evaluate line search candidates
            step = 1.0
            accepted = False
            for _ in range(self.step_halving): #perform step-halving back-tracking
line search to guarantee monotonic optimization steps
                beta_candidate = beta + step * delta
                new_obj = self._penalized_loglik(X_design, y, beta_candidate)
                if new_obj >= old_obj:
                    beta = beta_candidate
                    accepted = True
                    break
                step *= 0.5

            if not accepted: #fallback to a small static step if the backtracking
routine fails to improve the objective
                beta = beta + 0.1 * delta

            if np.max(np.abs(step * delta)) < self.tol: #terminate early if the
maximum absolute parameter change falls below convergence tolerance
                break

        self.intercept_ = np.array([beta[0]]) #export fitted model attributes
adhering to standard scikit-learn API conventions
        self.coef_ = beta[1:].reshape(1, -1)
        self.n_iter_ = iteration + 1
        self.classes_ = np.array([0, 1])
        return self

    def predict_proba(self, X):
        X = np.asarray(X, dtype=float) #validate array format and compute the
linear activation using stored parameters
        eta = self.intercept_[0] + X @ self.coef_.ravel()
        p = self._sigmoid(eta) #map activations to positive class probabilities
        return np.column_stack([1 - p, p]) #return a 2D array containing
probabilities for both negative and positive classes

    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int) #generate discrete
binary classifications using a standard 0.5 probability threshold
```

4. Train-test split

* Use train_test_split to define training and testing sets (X_train, X_train, y_train, y_test)
* Use the same split for all methods below
* Define an array of feature names (features)

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.35, stratify=y, random_state=7
) # split the dataset into stratified train and test subsets
features = X.columns # keep track of feature column names
```

```
print('Percentage of cases in train: {}'.format(np.sum(y_train)/len(y_train))) #
print the proportion of positive cases
```

```
Percentage of cases in train: 0.046153846153846156
```

5. Model without regularization

* Fit a logistic regression model without applying regularization. Use the following parameters for LogisticRegression:
* C = np.inf
* solver="saga"
* max_iter=10000
* tol=1e-6

* Assess performance by computing the AUC on the test set

* Write a function that takes as inputs:
* Xtrain
* y_train
* the class of the model (LogisticRegression in this case)
* an arbitrary number of keyword parameters, used to pass keyword parameters of the model

The function should perform a 5-fold random split of the training data and compute the model coefficients for each fold

* Write a function that takes as inputs the name of the features and the coefficients of the model computed by the previous function and that create a bar plot of the coefficients with the following properties:
* Features ordered by the absolute magnitude of their coefficients
* Display the mean and standard deviation
* Include a visual indication of the full range (minimum to maximum)

* What happens to the parameters of the model if you increase max_iter and decrease tol ? Why ?
* Do the important parameters for classification correspond to the expected ones ?

In [ ]:
```
sc = StandardScaler() # initialize the standard scaler
sc.fit(X_train) # compute mean and std from training data for scaling
X_train = sc.transform(X_train) # scale the training features
X_test = sc.transform(X_test) # scale the test features using training params
y_train = np.asarray(y_train) # convert training labels to numpy array
y_test = np.asarray(y_test) # convert test labels to numpy array
model = LogisticRegression(C = np.inf, solver="saga", max_iter=10000) # initialize
unpenalized logistic regression model
model.fit(X_train, y_train) # fit the model on scaled training data
prob = model.predict_proba(X_test)[:, 1] # predict probabilities for the positive
class
pred = (prob >= 0.5).astype(int) # apply 0.5 threshold to generate binary
classifications
print("AUC =", roc_auc_score(y_test, prob)) # calculate and print the roc auc
metric
```

```
AUC = 0.9621212121212122
```

In [ ]:
```
def estimate_parameters_range(X_train, y_train, model_class, **kwargs):
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # initialize
5-fold stratified cross-validation
    coefs = [] # create empty list to store coefficients from each fold
    for train_idx, val_idx in cv.split(X_train, y_train):
        X_tr = X_train[train_idx] # extract training features for the current fold
        y_tr = y_train[train_idx] # extract training labels for the current fold
        model = model_class(**kwargs) # instantiate the model with given
hyperparameters
        model.fit(X_tr, y_tr) # fit the model on the fold training data
        coef = model.coef_ # extract estimated model coefficients
        coefs.append(model.coef_.ravel()) # flatten and append coefficients to the
tracker list
    coefs = np.array(coefs) # convert the accumulated coefficients list into a
numpy array
    return coefs # return the array containing coefficients across all folds
```

In [ ]:
```
def plot_parameters_range(features, coefs):
    mean_coef = coefs.mean(axis=0) # calculate the mean coefficient value for each
feature
    min_coef = coefs.min(axis=0) # extract the minimum coefficient value across
folds
    max_coef = coefs.max(axis=0) # extract the maximum coefficient value across
folds
    std_coef = coefs.std(axis=0) # calculate the standard deviation of
coefficients
    order = np.argsort(np.abs(mean_coef))[::-1] # get sorting indices by
descending mean absolute magnitude
    mean_sorted = mean_coef[order] # sort mean coefficients according to
descending importance
    min_sorted = min_coef[order] # sort minimum coefficients according to feature
importance
    max_sorted = max_coef[order] # sort maximum coefficients according to feature
importance
    std_sorted = std_coef[order] # sort standard deviations according to feature
importance
    features_sorted = np.array(features)[order] # sort feature names to match the
ordered coefficients
    x = np.arange(len(features_sorted)) # generate tick positions for each sorted
feature axis
    plt.figure() # initialize a new figure object for the coefficient plot
    plt.bar(x, mean_sorted) # plot mean coefficient magnitudes as vertical bars
    plt.errorbar(x, mean_sorted, yerr=std_sorted, fmt='none', capsize=5) # add
standard deviation error bars
    for i in range(len(features_sorted)):
        plt.plot([i, i], [min_sorted[i], max_sorted[i]]) # plot vertical lines
indicating total range min-to-max
    plt.xticks(x, features_sorted, rotation=90) # apply sorted feature names to x-
```

```
axis ticks rotated 90 degrees
    plt.xlabel("Features (sorted by |coefficient|)") # set descriptive label for
the horizontal axis
    plt.ylabel("Coefficient value") # set descriptive label for the vertical axis
```

In [ ]:
```
coefs = estimate_parameters_range(X_train, y_train, LogisticRegression, C=np.inf,
solver="saga", max_iter=100000, tol=1e-8) # compute coefficients across folds
using unpenalized logistic regression
plot_parameters_range(features, coefs) # generate the error bar plot for the
estimated coefficient ranges
```

5. Model with L1, L2, and L1+L2 regularization

* Use 5-fold cross-validation to identify the optimal regularization paramter for L2 regularization (test the following values for C np.logspace(-4, 4, 20))
* Estimate the AUC of the best model in the testing set
* Plot the model parameters as before
* Repeat the previous points for L1 regularization, and elastic-net regularization (test the following values for l1_ratio \[0.1, 0.25, 0.5, 0.75, 0.9\])

In [ ]:
```
C_grid = np.logspace(-4, 4, 20) # define search grid of 20 logarithmic inverse
regularization strengths
model_l2 = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(solver="lbfgs", max_iter=10000))
]) # initialize machine learning pipeline with scaling and ridge logistic
regression
param_grid = {
    "logreg__C": C_grid
} # map the hyperparameter grid structure to the pipeline logistic regression step
grid_l2 = GridSearchCV(
    estimator=model_l2,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1,
    refit=True
) # configure 5-fold grid search cross-validation to optimize the roc auc metric
grid_l2.fit(X_train, y_train) # execute the cross-validation search grid over
training features and labels
best_idx = grid_l2.best_index_ # extract the array index corresponding to the
optimal configuration setup
mean_auc = grid_l2.cv_results_["mean_test_score"][best_idx] # retrieve validation
mean roc auc for the best model setup
std_auc  = grid_l2.cv_results_["std_test_score"][best_idx] # retrieve validation
standard deviation for the best model setup
print("Best C:", grid_l2.best_params_["logreg__C"]) # print out the optimized
hyperparameter value found
print("Mean CV AUC:", mean_auc) # print out the average cross-validation score
achieved
print("Std CV AUC:", std_auc) # print out the variation metric across validation
splits
best_model = grid_l2.best_estimator_ # extract the fully optimized pipeline fitted
on all training data
y_prob = best_model.predict_proba(X_test)[:, 1] # generate unseen test probability
targets using the best pipeline
test_auc = roc_auc_score(y_test, y_prob) # compute generalization performance on
unseen test data
print("Test AUC:", test_auc) # print out the performance evaluation metric for
test records
coefs = estimate_parameters_range(X_train, y_train, LogisticRegression,
C=grid_l2.best_params_["logreg__C"], solver="saga", max_iter=10000) # calculate
variance bounds for optimal parameters (warning: lacks scaler pipeline)
plot_parameters_range(features, coefs) # visualize structural feature weight
distributions and range deviations
```

In [ ]:
```
C_grid = np.logspace(-4, 4, 20) # define search grid of 20 logarithmic inverse
regularization strengths
model_l1 = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(l1_ratio=1.0, solver="saga", max_iter=10000))
]) # initialize machine learning pipeline with scaling and lasso/elasticnet
logistic regression
param_grid = {
    "logreg__C": C_grid
} # map the hyperparameter grid structure to the pipeline logistic regression step
grid_l1 = GridSearchCV(
    estimator=model_l1,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1,
    refit=True
) # configure 5-fold grid search cross-validation to optimize the roc auc metric
grid_l1.fit(X_train, y_train) # execute the cross-validation search grid over
training features and labels
best_idx = grid_l1.best_index_ # extract the array index corresponding to the
optimal configuration setup
mean_auc = grid_l1.cv_results_["mean_test_score"][best_idx] # retrieve validation
mean roc auc for the best model setup
std_auc  = grid_l1.cv_results_["std_test_score"][best_idx] # retrieve validation
standard deviation for the best model setup
print("Best C:", grid_l1.best_params_["logreg__C"]) # print out the optimized
hyperparameter value found
print("Mean CV AUC:", mean_auc) # print out the average cross-validation score
achieved
print("Std CV AUC:", std_auc) # print out the variation metric across validation
splits
best_model = grid_l1.best_estimator_ # extract the fully optimized pipeline fitted
on all training data
y_prob = best_model.predict_proba(X_test)[:, 1] # generate unseen test probability
targets using the best pipeline
test_auc = roc_auc_score(y_test, y_prob) # compute generalization performance on
unseen test data
print("Test AUC:", test_auc) # print out the performance evaluation metric for
test records
```

```
coefs = estimate_parameters_range(X_train, y_train, LogisticRegression,
C=grid_l1.best_params_["logreg__C"], l1_ratio=1.0, solver="saga", max_iter=10000)
# calculate variance bounds for optimal parameters (warning: lacks scaler pipeline
and penalty parameter)
plot_parameters_range(features, coefs) # visualize structural feature weight
distributions and range deviations
```

In [ ]:
```
C_grid = np.logspace(-4, 4, 20) # define search grid of 20 logarithmic inverse
regularization strengths
l1_ratio_grid = [0.1, 0.25, 0.5, 0.75, 0.9] # define the grid space for mixing
parameters between l1 and l2 penalties
model_en = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression(
        solver="saga",
        max_iter=10000
    ))
]) # initialize pipeline with scaling and elastic net logistic regression
(requires penalty="elasticnet")
param_grid = {
    "logreg__C": C_grid,
    "logreg__l1_ratio": l1_ratio_grid
} # map both regularizer coefficients and mixing ratio paths to the logreg
pipeline component
grid_en = GridSearchCV(
    estimator=model_en,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1,
    refit=True
) # configure 5-fold cross-validation grid search to optimize the double
hyperparameter space
grid_en.fit(X_train, y_train) # execute the cross-validation search grid over
training features and labels

best_idx = grid_en.best_index_ # extract the array index corresponding to the
optimal configuration setup
mean_auc = grid_en.cv_results_["mean_test_score"][best_idx] # retrieve validation
mean roc auc for the best model setup
std_auc  = grid_en.cv_results_["std_test_score"][best_idx] # retrieve validation
standard deviation for the best model setup
print("Best C:", grid_en.best_params_["logreg__C"]) # print out the optimized
hyperparameter value found
print("Best l1_ratio:", grid_en.best_params_["logreg__l1_ratio"]) # print out the
optimized mixing parameter value found
print("Mean CV AUC:", mean_auc) # print out the average cross-validation score
achieved
print("Std CV AUC:", std_auc) # print out the variation metric across validation
splits
best_model = grid_en.best_estimator_ # extract the fully optimized pipeline fitted
on all training data
y_prob = best_model.predict_proba(X_test)[:, 1] # generate unseen test probability
targets using the best pipeline
test_auc = roc_auc_score(y_test, y_prob) # compute generalization performance on
unseen test data
print("Test AUC:", test_auc) # print out the performance evaluation metric for
test records
coefs = estimate_parameters_range(X_train, y_train, LogisticRegression,
C=grid_en.best_params_["logreg__C"],
                    l1_ratio=grid_en.best_params_["logreg__l1_ratio"],
solver="saga", max_iter=10000) # calculate variance bounds for optimal parameters
(fixed grid object; warning: lacks scaler and penalty config)
plot_parameters_range(features, coefs) # visualize structural feature weight
distributions and range deviations
```

6. Model with Firth's regularization

* Use the class above to train a Firth's logistic regression model
* Plot the model parameters as before

In [ ]:
```
model = FirthLogisticRegression(max_iter=10000, tol=1e-8) # initialize the custom
Firth logistic regression model
model.fit(X_train, y_train) # fit the Firth regression model on the training data
prob = model.predict_proba(X_test)[:, 1] # predict probabilities for the positive
class on test data
pred = (prob >= 0.5).astype(int) # apply 0.5 threshold to generate binary
classifications
print("AUC =", roc_auc_score(y_test, prob)) # calculate and print the test roc auc
metric
```

```
AUC = 0.9734848484848485
```

In [ ]:
```
coefs = estimate_parameters_range(X_train, y_train, FirthLogisticRegression,
max_iter=100000, tol=1e-10) # compute coefficients across folds using custom Firth
logistic regression
plot_parameters_range(features, coefs) # generate the error bar plot for the
estimated Firth coefficient ranges
```

Expected conclusions

- The unregularized model often assigns a very large coefficient to the quasi-separating imaging flag.
- L2 regularization shrinks all coefficients and improves numerical stability, but generally keeps all predictors in the model.
- L1 regularization produces sparse models, but selected variables may be unstable under correlation.
- Firth's method reduces small-sample bias and moderates separation-driven coefficient inflation.
- The best AUC model is not necessarily the best calibrated or most interpretable model.

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Decision Trees and Ensemble Methods

Learning Objectives

1. Understand how Decision Trees perform classification
2. Train and visualize a Decision Tree classifier
3. Explain the role of pruning in preventing overfitting
4. Analyze instability in feature selection and tree structure
5. Improve classification performance using ensemble methods
6. Compare Decision Trees, Random Forests, and AdaBoost classifiers

1. Imports and Dataset

Run the following cell to import libraries and dataset. The dataset includes 30 features, related to the characteristics of a tumor tissue, and the corresponding classification of the tumor as benign or malignant:
* X = dataframe of input features
* y = class (0 = benign; 1 = tumor)
* features = the list of feature names
* class_names = the name of the two classes

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

# Load biomedical dataset
data = load_breast_cancer() # load raw breast cancer dataset structure

X = pd.DataFrame(data.data, columns=data.feature_names) # convert features into a
structured pandas dataframe
y = pd.Series(data.target, name="target") # convert target array into a pandas
series

features = X.columns # extract data feature column names
class_names = ['benign', 'tumor'] # map categorical class targets to explicit text
representations
print('Feature names:', features) # print out all available feature names
print('Number of samples:', X.shape[0]) # print out the total row count in the
dataset
print('Number of tumor samples:', np.sum(y == 1)) # print out the total number of
positive instances
```

2. Train/Test Split

* Split the dataset into train and test (30%)

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
) # split the dataset into stratified train and test subsets using a 70/30 ratio

print("Training samples:", X_train.shape[0]) # print the total number of samples
allocated to training
print("Testing samples:", X_test.shape[0]) # print the total number of samples
allocated to testing
```

```
Training samples: 398
Testing samples: 171
```

3. Complete Decision Tree

* Train a model of the class DecisionTreeClassifier without any pruning
* Evaluate its performance in the testing and training set
*Use the function plot_tree to visualize the decision tree. Usefull parameters to improve the quality of the figures are: filled, fontsize, feature_names, max_depth, and class_names. Another suggestion is to manually define the size of the figure in the command plt.figure (example: plt.*figure(figsize=(20, 10))*)*

In [ ]:
```
tree = DecisionTreeClassifier(random_state=42) # initialize decision tree model
with a fixed random seed
tree.fit(X_train, y_train) # train the decision tree classifier on training data

y_pred_train = tree.predict(X_train) # generate predictions for the training set
```

```
records
y_pred_test = tree.predict(X_test) # generate predictions for the testing set
records

print("Training accuracy:", accuracy_score(y_train, y_pred_train)) # evaluate and
print performance on training records
print("Testing accuracy:", accuracy_score(y_test, y_pred_test)) # evaluate and
print generalization performance on test records

plt.figure(figsize=(20, 10)) # initialize a wide canvas layout for tree structure
plotting
plot_tree(
    tree,
    feature_names=features,
    class_names=class_names,
    filled=True,
    max_depth=3,
    fontsize=8
) # render tree node structures up to the third level with colored condition
splits
plt.title("Decision Tree - First 3 Levels") # add a descriptive header to the
generated plot
plt.show() # display the final decision tree graph layout
```

4. Cost-Complexity Pruning

The hyperparameter `ccp_alpha` penalizes large trees.

The pruning objective is:

$$
R_\alpha(T) = R(T) + \alpha |T|
$$

where:

- R(T): classification error
- |T|: number of terminal nodes
- alpha: pruning strength

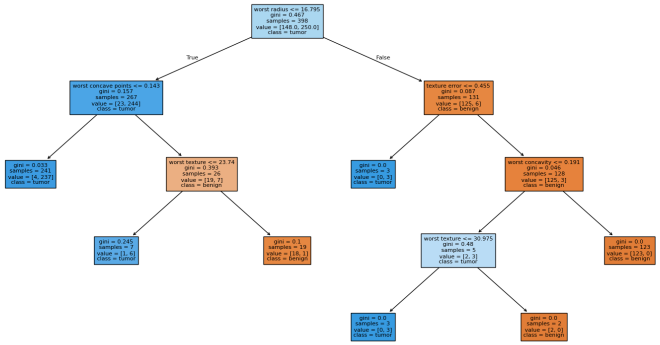
Larger values of `ccp_alpha` produce simpler trees.

* Use cross-validation to train a regularized decision tree. Test the following alpha values: \[0.0, 0.001, 0.005, 0.01, 0.02\]
* Report accuracy in the training and testing set
* Compare the regularized tree with the one defined before

In [ ]:
```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
) # initialize a 5-fold stratified cross-validation splitter with shuffling
param_grid = {
    "ccp_alpha": [0.0, 0.001, 0.005, 0.01, 0.02]
} # define candidate parameters for cost-complexity pruning to control overfitting
dt_cv = DecisionTreeClassifier(random_state=42) # instantiate a base decision tree
classifier with a fixed seed
grid_search = GridSearchCV(
    estimator=dt_cv,
    param_grid=param_grid,
    scoring="accuracy",
    cv=cv,
    n_jobs=-1,
    return_train_score=True
) # configure parallel grid search over alpha values tracking both train and test
scores
grid_search.fit(X_train, y_train) # execute the cross-validated parameter sweep on
the training set
print("Best cross-validation accuracy:", grid_search.best_score_) # print the
optimal average score achieved during validation
print("Best pruning parameters:", grid_search.best_params_) # print the specific
alpha parameter that yielded the highest accuracy
best_model = grid_search.best_estimator_ # extract the optimal pruned tree model
trained on the full training set
y_pred_train_pruned = best_model.predict(X_train) # generate predictions for the
training set using the best model
y_pred_test_pruned = best_model.predict(X_test) # generate predictions for the
testing set using the best model
print("Training accuracy:", accuracy_score(y_train, y_pred_train_pruned)) #
calculate and print accuracy on the training split
print("Testing accuracy:", accuracy_score(y_test, y_pred_test_pruned)) # calculate
and print generalization accuracy on the test split
```

```
Best cross-validation accuracy: 0.9421835443037974
Best pruning parameters: {'ccp_alpha': 0.005}
Training accuracy: 0.9849246231155779
Testing accuracy: 0.9298245614035088
```

In [ ]:
```
plt.figure(figsize=(20, 10))
plot_tree(
    best_model,
    feature_names=features,
    class_names=class_names,
    filled=True,
    fontsize=8
)
plt.show()
```



5. Instability of Decision Trees

Decision Trees are unstable learners.

Small changes in the training data may produce different tree structures and feature importances.

* Train two Decision Trees with parameter max_depth equal to 3 using different random splits of training and testing sets
* Plot the importance of the features (attribute .feature_importances_) for the two trees

In [ ]:
```
X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1,
    stratify=y
) # create first data split with seed 1 to demonstrate variance

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=10,
    stratify=y
) # create second data split with seed 10 to compare against the first

tree1 = DecisionTreeClassifier(max_depth=3, random_state=42) # initialize first
decision tree constrained to max depth 3
tree2 = DecisionTreeClassifier(max_depth=3, random_state=42) # initialize second
decision tree constrained to max depth 3

tree1.fit(X_train1, y_train1) # train the first model on the first training split
tree2.fit(X_train2, y_train2) # train the second model on the second training
split

f = plt.figure() # initialize a new figure object for the first tree's plot
ax1 = f.add_subplot(1,1,1) # add a single subplot axis to the figure
ax1.bar(np.arange(len(features)), tree1.feature_importances_) # plot first tree's
feature importances as vertical bars
ax1.set_xticks(np.arange(len(features))) # set x-axis tick positions corresponding
to each feature
ax1.set_xticklabels(features, rotation=90) # apply feature names as x-axis labels
rotated 90 degrees
plt.title('Tree1') # set a descriptive title for the first plot

f = plt.figure() # initialize a new figure object for the second tree's plot
ax1 = f.add_subplot(1,1,1) # add a single subplot axis to the figure
ax1.bar(np.arange(len(features)), tree2.feature_importances_) # plot second tree's
feature importances as vertical bars
ax1.set_xticks(np.arange(len(features))) # set x-axis tick positions corresponding
to each feature
ax1.set_xticklabels(features, rotation=90) # apply feature names as x-axis labels
rotated 90 degrees
plt.title('Tree2') # set a descriptive title for the second plot
```

6. Random Forests

Random Forests train many Decision Trees using:

- Bootstrap sampling
- Random feature subsets

For classification, the final prediction is usually obtained by majority vote.

The main idea is that combining many unstable trees creates a more stable and accurate classifier.

* Train a random forest classifier (class RandomForestClassifier) using the following parameters:
* n_estimators = 100
* max_depth = 5
* Report the accuracy in the training and testing set
* Repeat the previous analysis about the stability of features'importance in two separate training set

In [ ]:
```
rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
) # initialize a random forest classifier with 100 decision trees and a max depth
of 5
rf.fit(X_train, y_train) # train the random forest ensemble on the training data
split

y_pred_train_rf = rf.predict(X_train) # generate class predictions for the
training set records
```

```
y_pred_test_rf = rf.predict(X_test) # generate class predictions for the testing
set records

print("Training accuracy:", accuracy_score(y_train, y_pred_train_rf)) # calculate
and print performance metrics on the training set
print("Testing accuracy:", accuracy_score(y_test, y_pred_test_rf)) # evaluate and
print generalization accuracy on the unseen test set
```

```
Training accuracy: 0.992462311557789
Testing accuracy: 0.9415204678362573
```

In [ ]:
```
X_train1, X_test1, y_train1, y_test1 = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1,
    stratify=y
) # create first data split with seed 1 to evaluate ensemble variance

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=10,
    stratify=y
) # create second data split with seed 10 to compare feature weights

tree1 = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42) #
initialize first random forest ensemble model
tree2 = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42) #
initialize second random forest ensemble model

tree1.fit(X_train1, y_train1) # train the first forest ensemble on the first
training split
tree2.fit(X_train2, y_train2) # train the second forest ensemble on the second
training split

f = plt.figure() # initialize a new figure object for the first forest plot
ax1 = f.add_subplot(1,1,1) # add a single subplot axis to the figure layout
ax1.bar(np.arange(len(features)), tree1.feature_importances_) # plot feature
importances computed by the first forest model
ax1.set_xticks(np.arange(len(features))) # set x-axis tick positions corresponding
to each feature coordinate
ax1.set_xticklabels(features, rotation=90) # apply feature names as x-axis labels
rotated 90 degrees
plt.title('Tree1') # set a descriptive title for the first plot layout

f = plt.figure() # initialize a new figure object for the second forest plot
ax1 = f.add_subplot(1,1,1) # add a single subplot axis to the figure layout
ax1.bar(np.arange(len(features)), tree2.feature_importances_) # plot feature
importances computed by the second forest model
ax1.set_xticks(np.arange(len(features))) # set x-axis tick positions corresponding
to each feature coordinate
ax1.set_xticklabels(features, rotation=90) # apply feature names as x-axis labels
rotated 90 degrees
plt.title('Tree2') # set a descriptive title for the second plot layout
```

Each tree in the Random Forest has its own feature importance vector.

For each feature, we compute:

$$
SE_j = \frac{s_j}{\sqrt{M}}
$$

where:

- s_j is the standard deviation of feature importance across trees,
- M is the number of trees.

* Compute the average and the standard error of the feature importance. The trees are available from the attribute .estimators_ of the random forest object
* Plot the average feature importance with the corresponding standard error

In [ ]:
```
rf_tree_importances = np.array([
    estimator.feature_importances_
    for estimator in rf.estimators_
]) # extract feature importances from each individual decision tree in the random
forest ensemble

rf_mean_importance = rf_tree_importances.mean(axis=0) # compute the average
feature importance across all individual trees
rf_std_importance = rf_tree_importances.std(axis=0, ddof=1) # calculate the sample
standard deviation of feature importances
rf_se_importance = rf_std_importance / np.sqrt(rf_tree_importances.shape[0]) #
compute the standard error of the mean for feature importances

f = plt.figure() # initialize a new figure object for the ensemble feature
importance plot
ax1 = f.add_subplot(1,1,1) # add a single subplot axis to the layout structure
ax1.bar(np.arange(len(features)), rf_mean_importance, yerr= rf_se_importance ) #
plot mean importances as vertical bars with standard error lines
ax1.set_xticks(np.arange(len(features))) # set x-axis tick locations corresponding
to each feature coordinate
ax1.set_xticklabels(features, rotation=90) # apply feature names as x-axis labels
rotated 90 degrees
plt.title('Tree2') # set a descriptive title for the aggregate importance plot
```

17. Adaptive Boosting

AdaBoost sequentially trains weak learners, with the hey idea that misclassified samples receive larger weights, and consequenlty new classifiers focus more strongly on difficult cases. The final classifier combines weighted predictions from all weak learners.

* Train an Adaptive Boosting classifier (class AdaBoost) using the following parameters:

* n_estimators = 100
* learning_rate = 0.5
* Report the accuracy in the training and testing set

In [ ]:
```
ada = AdaBoostClassifier(
    n_estimators=100,
    learning_rate=0.5,
    random_state=42
) # initialize an adaboost classifier with 100 weak estimators and a 0.5 learning
rate
ada.fit(X_train, y_train) # train the adaboost ensemble model on the training data
split
y_pred_train_rf = ada.predict(X_train) # generate class predictions for the
training set records
y_pred_test_rf = ada.predict(X_test) # generate class predictions for the testing
set records
print("Training accuracy:", accuracy_score(y_train, y_pred_train_rf)) # calculate
and print performance metrics on the training set
print("Testing accuracy:", accuracy_score(y_test, y_pred_test_rf)) # evaluate and
print generalization accuracy on the unseen test set
```

```
Training accuracy: 1.0
Testing accuracy: 0.9532163742690059
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Support Vector Machines

In this exercise you will train Support Vector Machine classifiers on biomedical datasets and study two important model-selection questions:

* How should we select the regularization parameter C?
* How should we select the kernel?

1. Imports and Dataset

Run the following cell to import libraries and dataset. The dataset includes 30 features, related to the characteristics of a tumor tissue, and the corresponding classification of the tumor as benign or malignant:
* X = dataframe of input features
* y = class (0 = benign; 1 = tumor)
* features = the list of feature names
* class_names = the name of the two classes

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, balanced_accuracy_score,
precision_score, recall_score

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names) # convert features into a
structured pandas dataframe
y = pd.Series(data.target, name="target") # convert target array into a pandas
series

features = X.columns # extract data feature column names
class_names = ['benign', 'tumor'] # map categorical class targets to explicit text
representations
print('Feature names:', features) # print out all available feature names
print('Number of samples:', X.shape[0]) # print out the total row count in the
dataset
print('Number of tumor samples:', np.sum(y == 1)) # print out the total number of
positive instances
```

* Split into training and testing set

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42,
)
print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

```
Training set: (398, 30)
Test set: (171, 30)
```

2. SVM with linear kernel

* Define a Pipeline that includes the StandardScaler and an SVC model with liner kernel (why is is important to scale the data first ?)
* Perform grid search of the regularization parameter C over the values np.logspace(-4, 4, 9) using 5-fold cross validation. Use balanced_accuracy as scoring metric.
* Report the optimal value of the C parameter and the score of the best performing model
* Report accuracy, balanced accuracy, recall, and precision in the testing set

In [ ]:
```
linear_svm = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="linear")),
]) # initialize machine learning pipeline with feature scaling and a linear
support vector classifier
param_grid = {"svm__C": np.logspace(-4, 4, 9)} # define search grid of 9
logarithmic cost regularization parameters
grid_search = GridSearchCV(
    estimator=linear_svm,
    param_grid=param_grid,
    cv=5,
    scoring="balanced_accuracy",
    n_jobs=-1,
) # configure 5-fold cross-validation grid search optimizing for balanced accuracy
performance
grid_search.fit(X_train, y_train) # execute the cross-validation parameter sweep
on the training data split
print("Best C:", grid_search.best_params_["svm__C"]) # print out the optimal
regularization hyperparameter value found
```

```
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best
cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized linear svm
pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing
set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard
classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate
and print macro-averaged accuracy across classes
print("Recall:", recall_score(y_test, y_pred)) # calculate and print the true
positive rate metric on test records
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the
positive predictive value metric on test records
```

```
Best C: 0.01
Mean CV balanced accuracy: 0.9729885057471265
Accuracy: 0.9590643274853801
Balanced accuracy: 0.9484521028037383
Recall: 0.9906542056074766
Precision: 0.9464285714285714
```

3. SVM with polynomial kernel

* Repeat the previous point using a polynomial kernel. Test polynomial kernels of degrees 2 or 3.

In [ ]:
```
poly_svm = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="poly")),
]) # initialize machine learning pipeline with feature scaling and a polynomial
support vector classifier
param_grid = {"svm__C": np.logspace(-4, 4, 9), "svm__degree":[2, 3]} # define
hyperparameter grid covering cost regularization and polynomial degree targets
grid_search = GridSearchCV(
    estimator=poly_svm,
    param_grid=param_grid,
    cv=5,
    scoring="balanced_accuracy",
    n_jobs=-1,
) # configure 5-fold cross-validation grid search optimizing for balanced accuracy
performance
grid_search.fit(X_train, y_train) # execute the cross-validation parameter sweep
on the training data split
print("Best C:", grid_search.best_params_["svm__C"]) # print out the optimal
regularization hyperparameter value found
print("Best polynomial degree:", grid_search.best_params_["svm__degree"]) # print
out the optimal polynomial exponent degree selected
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best
cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized polynomial
svm pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing
set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard
classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate
and print macro-averaged accuracy across classes
print("Recall:", recall_score(y_test, y_pred)) # calculate and print the true
positive rate metric on test records
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the
positive predictive value metric on test records
```

```
Best C: 1000.0
Best polynomial degree: 3
Mean CV balanced accuracy: 0.9543218390804598
Accuracy: 0.9707602339181286
Balanced accuracy: 0.970356308411215
Recall: 0.9719626168224299
Precision: 0.9811320754716981
```

4. SVM with polynomial kernel

* Repeat the previous point using a gaussian kernel (rbf). Test the following values for the parameter gamma: np.logspace(-4, 1, 6).

In [ ]:
```
rbf_svm = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="rbf")),
]) # initialize machine learning pipeline with feature scaling and a radial basis
function support vector classifier
param_grid = {"svm__C": np.logspace(-4, 4, 9), "svm__gamma":np.logspace(-4, 1, 6)}
# define hyperparameter grid covering cost regularization and kernel coefficient
parameters
grid_search = GridSearchCV(
    estimator=rbf_svm,
    param_grid=param_grid,
    cv=5,
    scoring="balanced_accuracy",
    n_jobs=-1,
) # configure 5-fold cross-validation grid search optimizing for balanced accuracy
performance
grid_search.fit(X_train, y_train) # execute the cross-validation parameter sweep
on the training data split
print("Best C:", grid_search.best_params_["svm__C"]) # print out the optimal
regularization hyperparameter value found
print("Best gamma:", grid_search.best_params_["svm__gamma"]) # print out the
optimal radial basis function kernel width parameter
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best
cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized rbf svm
pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing
```

```
set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard
classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate
and print macro-averaged accuracy across classes
print("Recall:", recall_score(y_test, y_pred)) # calculate and print the true
positive rate metric on test records
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the
positive predictive value metric on test records
```

```
Best C: 10.0
Best gamma: 0.001
Mean CV balanced accuracy: 0.9729885057471265
Accuracy: 0.9707602339181286
Balanced accuracy: 0.9640771028037383
Recall: 0.9906542056074766
Precision: 0.9636363636363636
```

* Combine the 3 points above into a unique grid search that automatically select the best kernel and corresponding parameters

In [ ]:
```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC()),
]) # initialize machine learning pipeline with feature scaling and a default
support vector classifier
param_grid = [
    {
        "svm__kernel": ["linear"],
        "svm__C": np.logspace(-4, 4, 9),
    },
    {
        "svm__kernel": ["rbf"],
        "svm__C": np.logspace(-4, 4, 9),
        "svm__gamma": np.logspace(-4, 1, 6),
    },
    {
        "svm__kernel": ["poly"],
        "svm__C": np.logspace(-4, 4, 9),
        "svm__degree": [2, 3],
        "svm__gamma": ["scale"],
    },
] # define a comprehensive list of parameter grids covering distinct svm kernel
structures
grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="balanced_accuracy",
    cv=5,
    n_jobs=-1,
) # configure 5-fold cross-validation grid search to jointly optimize across all
target kernel spaces
grid_search.fit(X_train, y_train) # execute the comprehensive cross-validated
parameter search on the training split
print("Best parameters:") # print out the section label for the optimized model
settings
print(grid_search.best_params_) # print the specific optimal hyperparameter
dictionary found by the search sweep
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best
cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized multi-
kernel svm pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing
set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard
classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate
and print macro-averaged accuracy across classes
print("Recall:", recall_score(y_test, y_pred)) # calculate and print the true
positive rate metric on test records
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the
positive predictive value metric on test records
```

```
Best parameters:
{'svm__C': np.float64(0.01), 'svm__kernel': 'linear'}
Mean CV balanced accuracy: 0.9729885057471265
Accuracy: 0.9590643274853801
Balanced accuracy: 0.9484521028037383
Recall: 0.9906542056074766
Precision: 0.9464285714285714
```

5. SVM for a more complicated classification problem

The EEG Eye State dataset is a biomedical signal classification dataset derived from electroencephalography (EEG) recordings. The input features corresponds to 14 EEG signales, the output is a binary classification into open/closed eyes.

* Split data into training and test. Use 50% of data for the training (I suggest this splitting in order to reduce the number of samples in the training set, and consequenly the grid search operation. In case grid search is still too slow, decrease the number of samples in the training set).
* Compare the performances using a linear kernel and and rbf kernel. Use the following parameters:
* C = \[0.1, 1, 10\]
* gamma = \[0.01, 0.1\]

In [ ]:
```
from sklearn.datasets import fetch_openml

dataset = fetch_openml(
    name="eeg-eye-state",
    version=1,
    as_frame=True
)
X = dataset.data
y = dataset.target
y = y.astype(int)
```

```
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
Shape of X: (14980, 14)
Shape of y: (14980,)
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.50,
    stratify=y,
    random_state=42,
)
```

In [ ]:
```
print("Training set:", X_train.shape) # print out the dimensions and row count of
the training feature set
print("Test set:", X_test.shape) # print out the dimensions and row count of the
test feature set
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC()),
]) # initialize machine learning pipeline with feature scaling and a default
support vector classifier
param_grid = [
    {
        "svm__kernel": ["linear"],
        "svm__C": [0.1, 1, 10],
    },
    {
        "svm__kernel": ["rbf"],
        "svm__C": [0.1, 1, 10],
        "svm__gamma": [0.01, 0.1],
    },
] # define a condensed hyperparameter grid space comparing linear and rbf kernel
architectures
grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="balanced_accuracy",
    cv=5,
    n_jobs=-1,
) # configure 5-fold cross-validation grid search to jointly optimize across both
target kernel configurations
grid_search.fit(X_train, y_train) # execute the cross-validated parameter sweep
over the reduced search space
print("Best parameters:") # print out the section label for the optimized model
settings
print(grid_search.best_params_) # print the specific optimal hyperparameter
dictionary found by the search sweep
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best
cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized multi-
kernel svm pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing
set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard
classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate
and print macro-averaged accuracy across classes
#print("Recall:", recall_score(y_test, y_pred)) # temporarily commented out true
positive rate evaluation metric
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the
positive predictive value metric on test records
```

```
Training set: (7490, 14)
Test set: (7490, 14)
Best parameters:
{'svm__C': 10, 'svm__gamma': 0.1, 'svm__kernel': 'rbf'}
Mean CV balanced accuracy: 0.9303316991633629
Accuracy: 0.9448598130841122
Balanced accuracy: 0.9434299696149953
Precision: 0.943436754176611
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.

Open in Colab

SHAP Explainability for an SVM Classifier

This notebook demonstrates how to use **SHAP (SHapley Additive exPlanations)** to interpret the predictions of a nonlinear **Support Vector Machine (SVM)** with an **RBF kernel**.

The dataset used in this notebook is the OpenML dataset:

- **Dataset name:** `heart-disease`
- **OpenML data_id:** `43672`

The target variable is binary:

- `0` = no heart disease
- `1` = heart disease

SHAP explanations are computed on the probability estimates produced by the trained SVM classifier.

1. Import libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score, balanced_accuracy_score,
precision_score, recall_score
from sklearn.model_selection import GridSearchCV

import shap

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

2. Import Dataset

The code below:

- reads the dataset from a CSV file
- defines the categorical and numerical features
- creates:
- `X` : dataframe containing the input features
- `y` : target array (`1 = disease` , `0 = no disease`)
- `numerical_features` : list of numerical variables
- `categorical_features` : list of categorical variables

In [ ]:
```
d = pd.read_csv('data.csv')
categorical_features = ['sex', 'chest_pain_type', 'fasting_blood_sugar',
'resting_ecg', 'exercise_angina', 'ST_slope']
numerical_features = ['age', 'resting_bp_s', 'cholesterol', 'max_heart_rate',
'oldpeak']
X = d[categorical_features + numerical_features]
y = d['target'].values
```

3. Definition of the Machine Learning Pipeline

* Create a pipeline for numerical features that:
* imputes missing values using the median (Use SimpleImputer)
* standardizes features using `StandardScaler`
* mean = 0
* standard deviation = 1

* Create a second pipeline for categorical features that:
* imputes missing values using the most frequent category (Use SimpleImputer)
* applies one-hot encoding (Use OneHotEncoder)

* Combine the two preprocessing pipelines using `ColumnTransformer` , allowing different transformations to be applied to different feature groups

* Split the dataset into training and testing using stratified sampling to preserve the class distribution.

* Perform a grid search with 5-fold cross-validation to compare linear SVM and RBF-kernel SVM, using the following values for C = np.logspace(-4, 4, 9) and for gamma = np.logspace(-4, 1, 6)

In [ ]:
```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
]) # build pipeline to handle missing values via median substitution and normalize continuous scales

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
]) # build pipeline to fill missing categories with mode values and create dense dummy variables

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numerical_features),
    ("cat", categorical_transformer, categorical_features)
]) # combine specialized sub-pipelines to independently process numeric and categorical feature groups
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    stratify=y,
    random_state=RANDOM_STATE
) # split the full dataset into stratified train and test subsets using a 75/25 distribution ratio

print("Training set:", X_train.shape) # print out the dimensions and total row count of the training feature set
print("Test set:", X_test.shape) # print out the dimensions and total row count of the test feature set
```

```
Training set: (892, 11)
Test set: (298, 11)
```

In [ ]:
```
pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("svm", SVC(probability=True)),
]) # initialize machine learning pipeline integrating feature preprocessing and a probability-enabled support vector classifier
param_grid = [
    {
        "svm__kernel": ["linear"],
        "svm__C": [0.1, 1, 10], # ridotto da 9 a 3 valori
    },
    {
        "svm__kernel": ["rbf"],
        "svm__C": [0.1, 1, 10],      # ridotto da 9 a 3 valori
        "svm__gamma": [0.001, 0.01], # ridotto da 6 a 2 valori
    },
] # define parameter grids covering cost regularization and kernel coefficients across linear and rbf configurations
grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="balanced_accuracy",
    cv=5,
    n_jobs=-1,
) # configure 5-fold cross-validation grid search to jointly optimize across both target kernel spaces
grid_search.fit(X_train, y_train) # execute the cross-validated parameter search sweep on the preprocessed training split
print("Best parameters:") # print out the section label for the optimized model settings
print(grid_search.best_params_) # print the specific optimal hyperparameter dictionary found by the search sweep
print("Mean CV balanced accuracy:", grid_search.best_score_) # print the best cross-validation balanced accuracy score achieved
best_model = grid_search.best_estimator_ # extract the fully optimized preprocessing and svm pipeline fitted on all training data
y_pred = best_model.predict(X_test) # generate class predictions for the testing set records using the best model
print("Accuracy:", accuracy_score(y_test, y_pred)) # calculate and print standard classification accuracy on the test set
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred)) # evaluate and print macro-averaged accuracy across classes
print("Recall:", recall_score(y_test, y_pred)) # calculate and print the true positive rate metric on test records
print("Precision:", precision_score(y_test, y_pred)) # calculate and print the positive predictive value metric on test records
```

```
Best parameters:
{'svm__C': 10, 'svm__gamma': 0.01, 'svm__kernel': 'rbf'}
Mean CV balanced accuracy: 0.8513489606368865
Accuracy: 0.8590604026845637
Balanced accuracy: 0.8585443037974683
Recall: 0.8670886075949367
Precision: 0.8670886075949367
```

5. Create the SHAP Explainer

SHAP explanations require a **background dataset** representing the typical distribution of the training data.

Here, the background dataset is sampled from the **raw training dataframe**.

The explainer is then created using:

`python
best_model.predict_proba
`

This means SHAP calls the full fitted pipeline each time it needs a prediction:

1. raw input data are passed to the pipeline

2. the preprocessing step transforms the data
3. the SVM classifier returns class probabilities

The background dataset is used to estimate the baseline prediction $E[f(X)]$ which represents the average model output over the background samples.

In [ ]:
```
# Sample background data from the raw training dataframe
background = shap.sample(
    X_train,
    50,
    random_state=RANDOM_STATE
)

# Explain the full fitted pipeline directly
explainer = shap.Explainer(
    best_model.predict_proba,
    background
)
```

6. Compute SHAP Values

This section computes SHAP values for a subset of the raw test samples.

The code:

- selects the first test samples to explain
- passes the raw samples to the full fitted pipeline through the SHAP explainer
- computes feature contribution scores for each original input feature and each class

Because the explainer wraps the complete pipeline, the SHAP values are reported with respect to the original input columns rather than the one-hot encoded internal representation.

The resulting SHAP tensor usually has shape: $(number\_of\_samples,\ number\_of\_original\_features,\ number\_of\_classes)$

For example:

  `python
shap_values[0, :, 1]
  `

returns the SHAP values for:

- sample `0`
- all original input features
- class `1` (heart disease)

Positive SHAP values increase the probability of the selected class, while negative SHAP values decrease it.

In [ ]:
```
n_explain = 10

# Select raw test samples to explain
X_explain = X_test.iloc[:n_explain]

# Compute SHAP values using the full fitted pipeline
shap_values = explainer(X_explain)

print("SHAP values shape:", shap_values.values.shape)
print("Base values shape:", shap_values.base_values.shape)
print("Explained data shape:", X_explain.shape)
```

```
PermutationExplainer explainer: 11it [00:14,  2.38s/it]


SHAP values shape: (10, 11, 2)
Base values shape: (10, 2)
Explained data shape: (10, 11)
```

7. Global Feature Importance with a Beeswarm Plot

The beeswarm plot provides a global view of feature importance across the explained samples.

This section:

- extracts SHAP values corresponding to the positive class (`class 1`)
- creates a `shap.Explanation` object for the positive class
- uses the original feature values and feature names
- visualizes the distribution of SHAP values using a beeswarm plot

Interpretation:

- features are ordered by average importance
- each point corresponds to one sample
- horizontal position represents the SHAP contribution
- red points generally indicate high feature values
- blue points generally indicate low feature values

Because the full pipeline is explained directly, the displayed features correspond to the original dataset columns, not the internal one-hot encoded columns.

In [ ]:
```
positive_class_index = 1 # define the target class index for calculating feature
importance contributions

shap_values_positive = shap.Explanation(
    values=shap_values.values[:, :, positive_class_index],
    base_values=shap_values.base_values[:, positive_class_index],
    data=X_explain.values,
    feature_names=X_explain.columns.tolist()
) # extract and bundle multi-class shapley arrays into a single-class explanation
object wrapper

shap.plots.beeswarm(shap_values_positive) # generate a beeswarm plot to visualize
feature distribution impacts on the target class
```

8. Local Explanation with a Waterfall Plot

The waterfall plot explains the prediction for a single sample.

This section:

- selects one raw test sample from the explained subset
- extracts the SHAP values for the positive class
- builds a single-sample `shap.Explanation` object
- visualizes how each original feature contributes to the final predicted probability

The waterfall plot starts from the baseline prediction $E[f(X)]$ and then shows how individual feature contributions push the prediction until reaching the final probability predicted by the full pipeline.

In [ ]:
```
sample_idx = 1 # set the row index of the specific instance to evaluate
positive_class_index = 1 # specify the positive class index to isolate
contribution values

single_sample_explanation = shap.Explanation(
    values=shap_values.values[sample_idx, :, positive_class_index],
    base_values=shap_values.base_values[sample_idx, positive_class_index],
    data=X_explain.iloc[sample_idx].values,
    feature_names=X_explain.columns.tolist()
) # extract shapley value metrics for a single row into a standalone explanation
object

shap.plots.waterfall(single_sample_explanation) # generate a waterfall chart
tracking the cumulative feature impacts for this instance
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.